

Cycle Agent

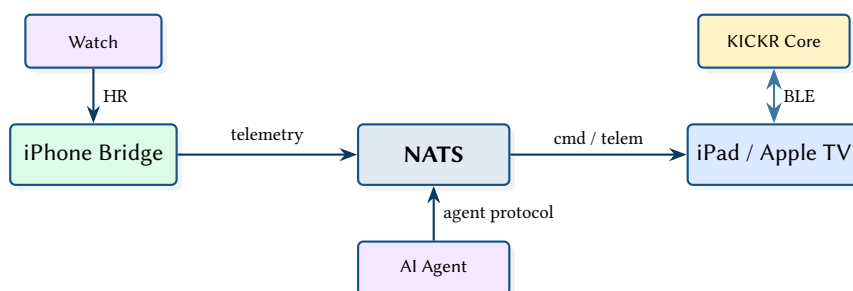
An Agent-Bridged Architecture for AI-Driven Indoor Cycling on Apple Platforms

Simon Knight, Ph.D.

Adelaide, Australia

March 2026 • Version 0.1.0

Last updated: March 16, 2026



Abstract. Cycle Agent is a native SwiftUI application for iPad and Apple TV that bridges professional cycling hardware (Wahoo KICKR Core) with dynamic AI-driven workout logic through NATS messaging, visualized in a procedurally generated SceneKit environment. This report describes the architecture of the system, focusing on three key design decisions: (1) a hub-and-spoke NATS broker that decouples the display surface from the agent brain, (2) a multi-app topology where an iPhone bridge relays biometric data from Apple Watch to the training display without requiring direct connectivity, and (3) a strictly typed protocol envelope that enables forward-compatible schema evolution. We present the current state of the implementation, discuss architectural invariants and their rationale, and identify gaps and offline preparation opportunities for the remaining development phases.

Contents

I ARCHITECTURE

1 The Indoor Cycling Problem	2
2 System Topology	2
2.1 Why NATS.	2
3 The Four Roles	3
3.1 Host App: Display and Hardware Control.	3
3.2 Bridge App: The iPhone Relay	3
3.3 Watch App: Sensor and Recorder.	3
3.4 AI Agent: The Workout Brain	3
4 Protocol Design	4
4.1 Agent Protocol	4
4.2 Health Protocol.	4
5 BLE and FTMS Integration	4
5.1 Connection Lifecycle	4
5.2 FTMS Data and Control	4
6 Rendering: The World	5
6.1 Chunk-Based Terrain	5
6.2 Visual Speed Mapping.	5
6.3 Camera System	6
7 Heart Rate Processing	6

II ANALYSIS

8 What Works	6
9 Gaps and Open Questions	6
9.1 No Offline Agent Strategy.	6
9.2 No Authentication or Multi-Tenancy	7
9.3 No Offline Workout Execution	7
9.4 tvOS Focus Engine	7
9.5 No User Profile or Zone Calibration.	7
9.6 No Telemetry Persistence or Replay.	7
9.7 No BLE Error Budget or Recovery Strategy.	7
9.8 No Watch-Bridge Crash Recovery.	7
9.9 No Power Calibration or Spindown	7
10 Consistency and Staleness	8
11 The Control Design Space	8
12 Failure Modes	8
13 Offline Preparation Opportunities	9
13.1 Workout Schema and Library.	9
13.2 Agent Decision Engine	10
13.3 Terrain Themes and Biomes	10
13.4 Ride Recording and Export.	10
13.5 Protocol Conformance Test Suite	10
13.6 Onboarding and Profile UI Prototypes	10
14 Architectural Invariants	10
15 Future Directions	10
15.1 Multi-Rider Sessions	10
15.2 Adaptive Terrain from Agent State.	11
15.3 Voice Coaching via Agent Telemetry	11
15.4 Recorded Ride Ghosts	11
15.5 ERG Mode Support	12
15.6 Local Agent Fallback.	12

16	Product and Feature Ideas	12
16.1	Workout Builder UI	12
16.2	Training Plan Progression.	13
16.3	Live Metrics Dashboard via NATS	13
16.4	Social Riding and Leaderboards	13
16.5	Post-Ride Analytics Report.	13
16.6	Zwift/TrainerRoad Course Import	14
16.7	Music Sync Agent	14
17	Experiential and Sensory Ideas	14
17.1	Haptic Feedback on Zone Transitions	15
17.2	Time-of-Day Lighting Cycle	15
17.3	GPX Route Replay.	15
17.4	Metal Shader Effects	15
17.5	Siri and Shortcuts Integration.	16
17.6	SharePlay Group Rides	16
18	Data and Intelligence Ideas	16
18.1	Critical Power Model	16
18.2	Cardiac Drift Detection.	17
18.3	Perceived Exertion Prediction.	17
18.4	Digital Twin / Simulation Mode	17
18.5	Third-Party Agent Marketplace	18
18.6	Observability via NATS Subjects	19
18.7	Standalone Power Meter Support.	19
18.8	ANT+ Bridge	19
19	Accessibility and Inclusivity	19
19.1	VoiceOver and Periodic Announcements.	20
19.2	Dynamic Type and High Contrast	20
19.3	Reduced Motion Mode	20
19.4	Switch Control and External Devices.	20
20	Device Integration Ideas	20
20.1	Smart Fan Control.	20
20.2	External Display / Stage Manager	20
20.3	Picture-in-Picture on iPad.	20
20.4	Apple Watch Complications and Live Activities	21
20.5	iCloud Sync via CloudKit	21
21	Advanced Metrics and Training Science	21
21.1	Power Smoothing Modes	21
21.2	Normalised Power and Training Stress	21
21.3	Lap and Segment Tracking	22
21.4	Pedal Balance and Efficiency	22
21.5	Cadence-Based Coaching	22
21.6	Readiness Score from HealthKit	22
21.7	Real-Time Power Curve Comparison.	22
22	Gamification and Engagement	23
22.1	Achievement System.	23
22.2	Terrain Theme Unlocks.	23
22.3	Strava Live Segment Integration.	23
23	Real-World Geography	23
23.1	Elevation Data: SRTM and Open Topography	23
23.2	OpenStreetMap Road Geometry.	23
23.3	MapKit and Apple Maps Overlays	24
23.4	Famous Climb Database	24
23.5	Offline Tile Preparation.	24
23.6	Terrain Fidelity Levels.	25
23.7	Street View Panoramas and Video Routes	25
24	Performance Considerations	25
24.1	NATS Message Throughput	26

24.2	Memory Pressure from Terrain Chunks	26
24.3	Apple Watch Battery Impact	26
24.4	CADisplayLink vs. Timer Render Loop	26
25	Summary	26
	Revision History	27
BLE	Bluetooth Low Energy	2
FTMS	Fitness Machine Service	4
HR	Heart Rate	3
EMA	Exponential Moving Average	6

ARCHITECTURE

1 The Indoor Cycling Problem

Indoor cycling applications face a fundamental integration challenge. The trainer hardware speaks Bluetooth Low Energy (BLE); the AI agent that decides what workout to prescribe may run anywhere from a local process to a cloud service; the display surface is a large-screen device (iPad or Apple TV) optimised for glanceable metrics at speed; and biometric sensors live on the rider's wrist. These four roles have incompatible lifecycle characteristics: the trainer must maintain a persistent radio link, the agent must maintain conversational state, the display must sustain 60 fps rendering, and the watch must record a continuous workout session even if the display app is suspended.

Most indoor cycling platforms collapse these roles into a monolithic application. The result is a system where adding a new agent strategy, or moving the display to a different device, or swapping the biometric source requires rearchitecting the entire application.

Cycle Agent takes a different approach. The system decomposes into four independent roles connected by a message broker, and each role maps to a distinct hardware device.

Insight:
The display surface, the intelligence, and the hardware form a triangle of concerns that should never be collapsed into a single process.

: Role-Device Separation

Each architectural role (Display, Bridge, Sensor, Agent) maps to exactly one device process. No single process combines more than one role. All inter-role communication flows through the NATS broker.

2 System Topology

The system follows a hub-and-spoke topology with NATS at the centre. Figure 1 shows the four device roles and their communication paths.

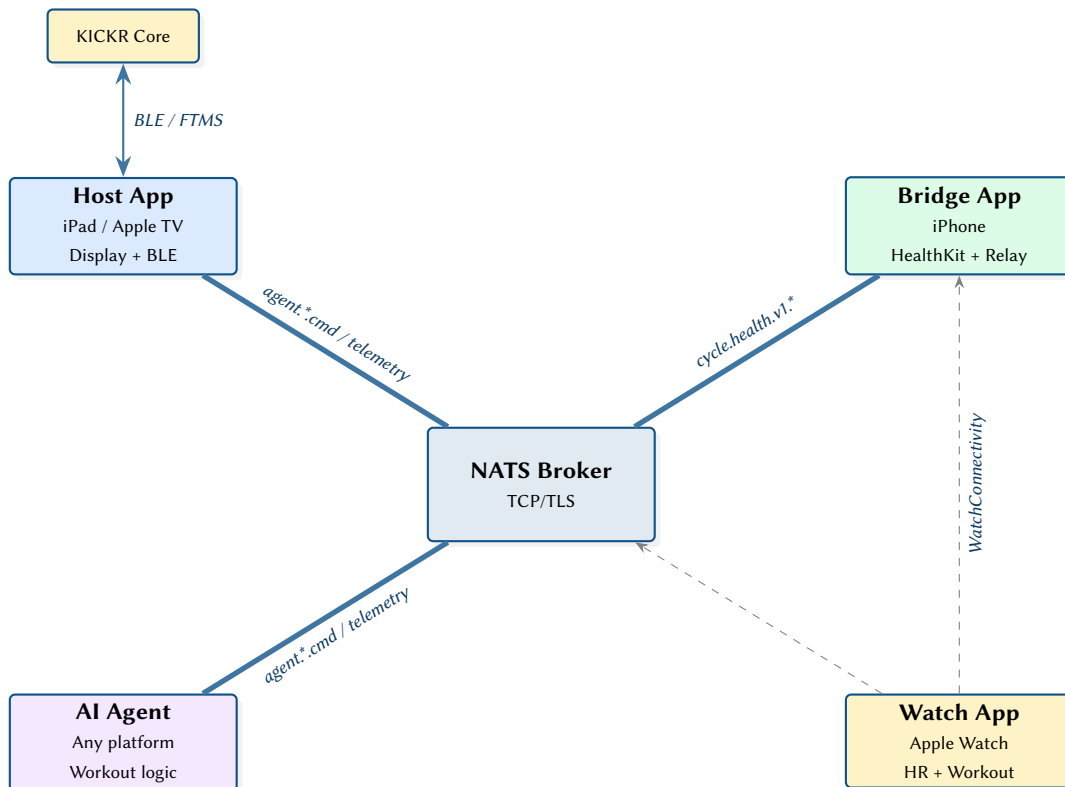


Figure 1. Hub-and-spoke topology. Every inter-role message flows through NATS except two links: the Watch-to-iPhone relay (WatchConnectivity) and the Host-to-Trainer radio link (BLE/FTMS). These exceptions exist because Apple Watch cannot maintain an independent TCP connection to an external broker, and BLE is inherently point-to-point.

2.1 Why NATS

The choice of NATS as the broker is deliberate. The agent protocol requires request-response semantics for commands ("set the grade to 5%") and pub/sub semantics for telemetry ("the agent is currently targeting 7%").

Insight:
NATS provides both request-response and pub/sub semantics in a single lightweight binary with zero configuration.

grade”). NATS provides both patterns natively. It runs as a single binary with no external dependencies, supports TLS, and has an official Swift client with full `async/await` support.

The alternative—embedding the agent logic directly in the app, or using HTTP polling, or using WebSockets—would couple the agent’s lifecycle to the display app. With NATS, the agent can crash and restart, the display can background and resume, and a second display can subscribe to the same telemetry stream without any coordination.

3 The Four Roles

3.1 Host App: Display and Hardware Control

The Host App runs on iPad or Apple TV. It is responsible for two concerns that are intentionally co-located: rendering the visual environment and maintaining the BLE connection to the trainer.

: BLE Locality

The Bluetooth radio link to the KICKR Core must originate from the same device that renders the primary display. This eliminates one network hop from the latency-critical resistance control path.

The Host discovers and connects to the trainer using CoreBluetooth with state preservation and restoration. When the system terminates the app in the background, CoreBluetooth can relaunch it and restore the peripheral connection without a full re-scan. This is critical on tvOS, where the app may be suspended during screensaver activation.

The rendering subsystem produces an infinite procedurally generated terrain through which the rider visually travels. The terrain is chunk-based: as the rider’s virtual position advances, new terrain segments are generated ahead and old ones are recycled. Visual speed is derived from instantaneous power using a simplified physics model:

$$v(P) = \sqrt{\frac{P}{k}} \quad \text{where } k = 3.0 \text{ kg/m}$$

This is not a physics simulation. It is a perceptual mapping chosen to produce a satisfying visual response across the 50–350 W range typical of indoor cycling.

3.2 Bridge App: The iPhone Relay

The Bridge App runs on the rider’s iPhone. Its sole purpose is to receive biometric data from the Watch and relay it to the NATS broker.

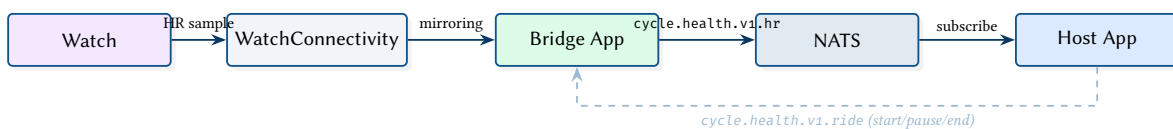


Figure 2. Heart rate relay path. The Watch cannot speak NATS directly; the iPhone bridge translates WatchConnectivity messages into NATS publishes. A reverse path carries ride lifecycle commands back to the bridge for HealthKit workout recording.

The Bridge also listens for ride lifecycle commands on the reverse path. When the Host App starts a workout, the Bridge receives a `ride.start` command and begins a HealthKit workout session. When the ride ends, the Bridge commits the workout to Apple Health. This design means the iPad and Apple TV—neither of which have HealthKit on their respective platforms—can still trigger persistent health records.

3.3 Watch App: Sensor and Recorder

The Watch App has two responsibilities: sampling heart rate during a workout and recording the workout session in HealthKit. It communicates exclusively with the iPhone via WatchConnectivity’s application context and message-passing APIs.

The Watch does not know about NATS, the trainer, or the agent. It receives a “start workout” signal from the Bridge, begins an `HKWorkoutSession`, samples Heart Rate (HR) via an `HKAnchoredObjectQuery`, and mirrors each sample to the iPhone.

3.4 AI Agent: The Workout Brain

The Agent is the only role that is not an Apple-platform application. It connects to NATS, subscribes to telemetry from the Host (power, cadence, speed), and publishes commands (target grade) and its own telemetry (reasoning state, current target). The Agent protocol is versioned and schema-typed, meaning the Agent can be written in any language with a NATS client.

Insight:
The Bridge exists because Apple Watch cannot maintain an independent TCP socket to an external broker. This is a platform constraint, not an architectural preference.

Design Decision 1: Agent as External Process

The Agent is deliberately external. It could be a Python script, a Rust binary, a cloud function, or a locally running LLM. The only contract is the NATS subject namespace and the JSON envelope schema. This decoupling means the training intelligence can evolve independently of the display and hardware layers.

4 Protocol Design

All NATS messages use a versioned JSON envelope. This is the single most important interoperability decision in the system.

: Versioned Envelopes

Every NATS message is wrapped in an `AgentEnvelope<T>` or `HealthEnvelope<T>` carrying a `schemaVersion` integer. Receivers must reject envelopes with an unrecognised schema version rather than attempting partial parsing.

4.1 Agent Protocol

The agent protocol defines two subject patterns:

- `cycle.agent.{agentId}.cmd` — Request-response. The Host sends a command (ping, set target grade); the Agent replies with an acknowledgement and optional message.
- `cycle.agent.{agentId}.telemetry` — Pub/sub. The Agent publishes its current state (target grade, reasoning notes) at its own cadence. The Host subscribes and updates the display.

The `agentId` segment allows multiple agents to coexist on the same broker. A training agent and a coaching agent could operate simultaneously, each on its own subject namespace.

4.2 Health Protocol

The health protocol defines two subjects:

- `cycle.health.v1.hr` — Pub/sub. The Bridge publishes heart rate samples from the Watch. The Host subscribes.
- `cycle.health.v1.ride` — Pub/sub. The Host publishes ride lifecycle events (start, pause, resume, end). The Bridge subscribes and drives HealthKit accordingly.

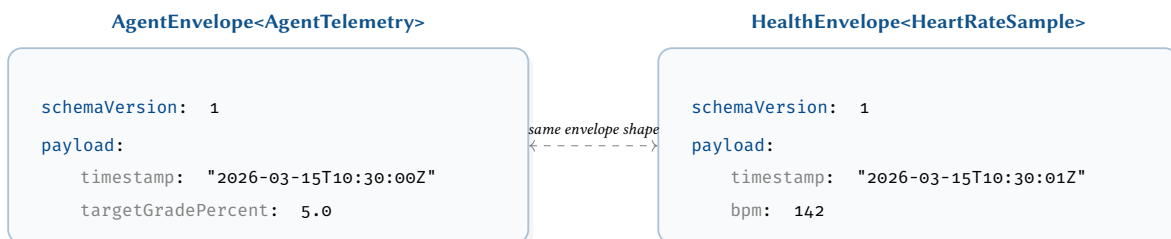


Figure 3. Both protocol families share the same envelope structure: a schema version integer and a typed payload. The encoder and decoder are reused across both protocols, ensuring consistent ISO 8601 date formatting and forward-compatible parsing.

5 BLE and FTMS Integration

The BLE layer communicates with the KICKR Core trainer using the Fitness Machine Service (FTMS) standard defined in the Bluetooth SIG specification. This is not a proprietary Wahoo protocol; any FTMS-compliant trainer should work.

5.1 Connection Lifecycle

The system uses CoreBluetooth’s state preservation and restoration mechanism. When the app registers a restoration identifier, the system can relaunch it after background termination and call `willRestoreState` with previously connected peripherals. The BLEManager must immediately cache strong references to restored peripherals; failure to do so allows ARC to deallocate them before the connection can be re-established.

5.2 FTMS Data and Control

The trainer exposes two primary characteristics:

1. **Indoor Bike Data** (notify): A packed binary payload containing instantaneous speed, cadence, and power. The system parses this using a bit-flag-driven parser that reads only the fields indicated by the flags word.

Insight:
The transition from Connected to Ready is gated on CCCD indication enablement. Without this step, Control Point writes will silently fail—the trainer accepts the data but sends no response.

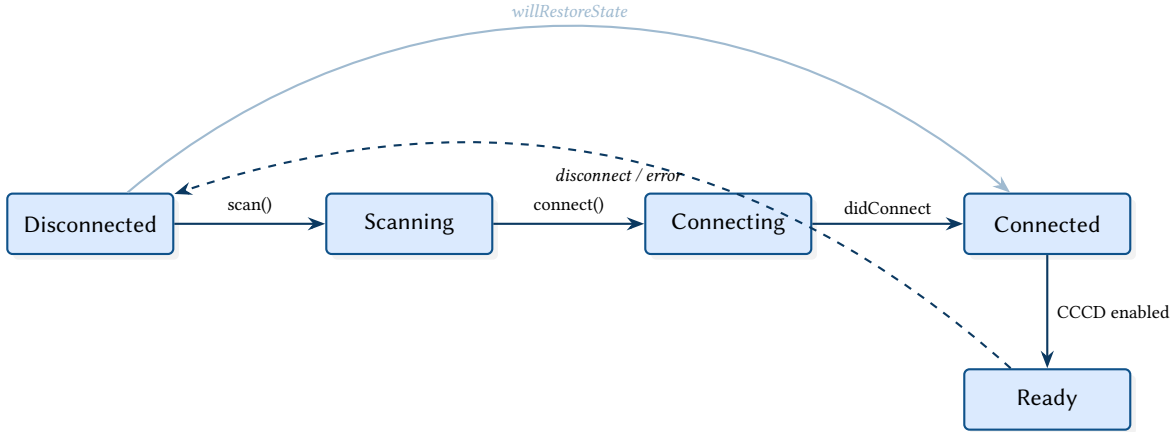


Figure 4. BLE connection state machine. The “Ready” state requires both a successful connection and CCCD indication enablement on the Control Point characteristic. The dashed restore path handles system-initiated relaunch after background termination.

2. **Control Point** (write + indicate): Accepts simulation parameters (grade, wind speed, rolling resistance, wind resistance coefficient) and responds with a result code via indication. Writes are serialized using Swift’s CheckedContinuation to prevent overlapping requests.

The parsed metrics pass through a smoothing buffer before reaching the UI. The buffer uses a sliding window with configurable depth, defaulting to 3 samples. This eliminates the visual jitter common in raw BLE telemetry without introducing perceptible latency.

6 Rendering: The World

The visual environment is a procedurally generated low-poly landscape rendered in SceneKit. The system does not attempt photorealism; it targets a stylised aesthetic that is legible from a 10-foot viewing distance on Apple TV.

6.1 Chunk-Based Terrain

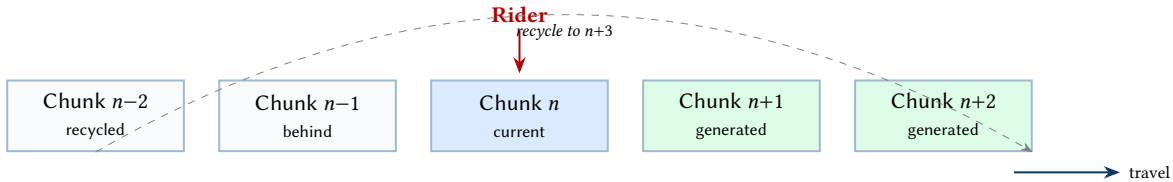


Figure 5. Chunk lifecycle. As the rider advances, the oldest chunk behind the camera is recycled: its geometry is regenerated with the seed for position $n+3$ and moved to the front of the queue. At any time, five chunks are alive, giving approximately 500 m of visible terrain.

Each chunk is a flat grid of vertices displaced vertically using simplex noise seeded deterministically from the chunk’s position index and a global scene seed. The deterministic seeding means the same ride will always produce the same terrain, enabling replay and screenshot consistency.

6.2 Visual Speed Mapping

The visual speed mapper converts instantaneous power into a travel speed through the terrain. The mapping uses a square-root relationship:

$$v = \sqrt{\frac{P}{3.0}} \text{ m/s}$$

This is not derived from cycling physics. A realistic model would require rider weight, drag coefficient, and rolling resistance—parameters that are unavailable without user profile data. The square-root mapping was chosen because it compresses the perceptual range: a rider at 100 W sees meaningful visual movement, while a rider at 300 W sees noticeably faster terrain without the speed becoming disorienting.

6.3 Camera System

The camera follows the rider using a “chase camera” rig: a parent harness node whose position tracks the rider, with the camera offset behind and above. The harness position is interpolated rather than snapped, producing a smooth following motion that absorbs instantaneous speed changes.

7 Heart Rate Processing

Heart rate data arrives as discrete BPM samples from the Watch, relayed through the Bridge via NATS. The Host processes these samples through a `HeartRateModel` that provides three derived signals:

1. **Smoothed BPM:** An Exponential Moving Average (EMA) with $\tau = 3.5$ s removes noise from optical sensor jitter without masking genuine cardiovascular response.
2. **Zone:** The smoothed BPM is classified into five standard training zones (Recovery, Endurance, Tempo, Threshold, VO_2 Max) using configurable thresholds.
3. **Trend:** A time-windowed linear regression over the last 15 s classifies the heart rate trajectory as rising, flat, or falling.

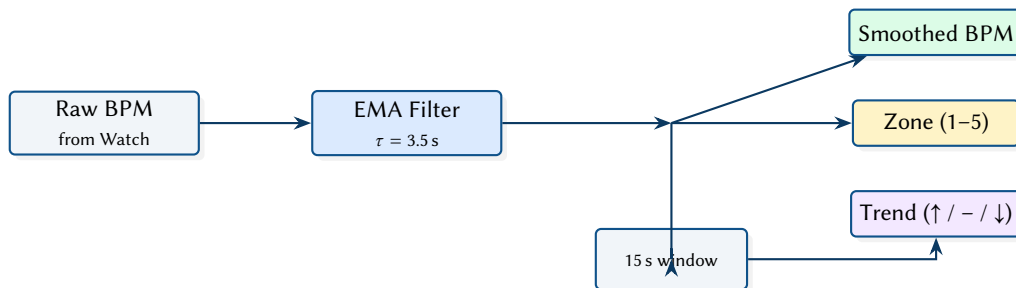


Figure 6. Heart rate processing pipeline. The EMA filter produces a smoothed signal; zone classification and trend detection operate downstream. The trend detector maintains its own 15-second sample window, independent of the EMA state.

ANALYSIS

8 What Works

Three architectural decisions have proven their value across the first four development phases.

The NATS broker decouples everything. The Agent can be developed and tested without any Apple hardware. A mock agent executable is included in the repository; it connects to NATS, receives pings, and publishes synthetic telemetry. This mock has been the primary development companion since Phase 2, and it runs on macOS without Xcode.

The SwiftPM library contains no @main. The entire CycleAgent module is a library target. The three app targets (Host, Bridge, Watch) embed it. This means `swift test` works on macOS for all non-UI, non-BLE logic: protocol encoding, FTMS parsing, heart rate modelling, metrics smoothing. No simulator required.

Typed protocol envelopes prevent silent mismatches. Because every message carries a schema version, adding a new field to `AgentTelemetry` does not break old consumers. They decode the envelope, see the known schema version, and ignore unknown keys via standard Codable behaviour. If the schema changes in a breaking way, the version increments and old consumers reject explicitly.

9 Gaps and Open Questions

9.1 No Offline Agent Strategy

The agent protocol defines the transport, but no reference agent exists beyond the mock. The mock publishes a fixed grade on a timer; it does not respond to power data, does not implement interval logic, and does not adapt to heart rate. A real training agent needs:

- A workout model (intervals, ramps, steady state, recovery).
- Real-time adaptation based on power and heart rate drift.
- A persistence layer for workout history and progression.

This is the largest functional gap. The protocol is ready; the intelligence is not.

Insight:
The EMA time constant of 3.5 s was chosen empirically. Shorter values track sensor noise; longer values mask the cardiac response to interval efforts. This value tracks a 30-second interval start within 2–3 heartbeats.

9.2 No Authentication or Multi-Tenancy

NATS supports authentication (tokens, NKeys, JWTs) but the current implementation connects without credentials. In a shared-network environment, any NATS client can subscribe to any subject. For a single-user home network this is acceptable; for a gym or shared trainer scenario, it is not.

9.3 No Offline Workout Execution

If the NATS broker is unreachable, the Host App cannot receive agent commands. The system has no local fallback: no cached workout plans, no embedded interval logic, no ability to execute a pre-programmed session without connectivity. The rider can still pedal and see metrics, but the training intelligence is absent.

9.4 tvOS Focus Engine

The current UI uses standard SwiftUI controls. On Apple TV, navigation requires the Siri Remote and the tvOS focus engine. The resistance slider, device list, and settings views have not been tested or optimised for focus-based navigation. This is Phase 5 work, but it may require structural changes to the view hierarchy if focusable regions conflict.

9.5 No User Profile or Zone Calibration

Heart rate zones are currently hardcoded to generic percentage thresholds. A real training application needs FTP (Functional Threshold Power), maximum heart rate, and resting heart rate to compute personalised zones. There is no onboarding flow, no user profile, and no way to calibrate thresholds.

9.6 No Telemetry Persistence or Replay

All NATS messages are fire-and-forget. If the Host App is backgrounded for three seconds and misses four heart rate samples during that window, those samples are gone. There is no message journaling, no replay buffer, and no way for a consumer to request “everything I missed since timestamp t .” NATS JetStream provides exactly this capability—durable streams with consumer acknowledgement and replay—but the current implementation uses core NATS only.

This gap becomes critical when ride recording is implemented. The recording layer must either (a) subscribe via JetStream with durable delivery, or (b) record on the Bridge (which is always awake) and sync the complete trace after the ride. Both approaches require architectural changes.

Insight:
Fire-and-forget is acceptable for real-time display, where a stale sample has no value. It is not acceptable for ride recording, where every sample contributes to the historical record.

9.7 No BLE Error Budget or Recovery Strategy

The BLE state machine (fig. 4) handles connection and disconnection, but does not define a recovery policy. When the trainer drops mid-interval—due to radio interference, trainer firmware crash, or the rider walking out of range—the system transitions to Disconnected and stays there. There is no automatic reconnection attempt, no exponential backoff, and no notification to the agent that its grade commands are being silently dropped.

A robust system needs:

- **Reconnection policy:** Automatic scan-and-reconnect with configurable backoff (e.g., 1 s, 2 s, 4 s, cap at 15 s).
- **Agent notification:** A “hardware lost” telemetry event so the agent can pause the workout rather than continuing to prescribe efforts the rider cannot execute.
- **Stale command detection:** If a grade command was queued before the disconnect, it must be discarded rather than replayed after reconnection, as the workout context may have changed.

9.8 No Watch-Bridge Crash Recovery

The Watch and Bridge communicate via WatchConnectivity, which is inherently best-effort. If the Bridge crashes mid-workout, the Watch continues recording its HealthKit session—this is correct—but the heart rate relay to the Host stops. The rider sees a frozen heart rate display with no indication that the data source has failed.

When the Bridge relaunches, it has no mechanism to:

1. Discover that a workout is already in progress on the Watch.
2. Request the backlog of heart rate samples missed during the crash.
3. Resume the NATS relay without the Host issuing a new `ride.start` command.

9.9 No Power Calibration or Spindown

FTMS trainers drift over time and temperature. Wahoo recommends a spindown calibration before each session: the rider accelerates to a target speed, stops pedalling, and the trainer measures the deceleration curve to calibrate its power reading. The FTMS specification includes a Spindown Control procedure (opcode 0x01 on the Control Point), but Cycle Agent does not implement it. Riders must currently use the Wahoo app for

calibration, then switch to Cycle Agent for the workout. This is a friction point that breaks the single-app experience.

10 Consistency and Staleness

Every value displayed on the Host screen is stale. The question is not whether the data is delayed, but by how much, and whether the delay is bounded.

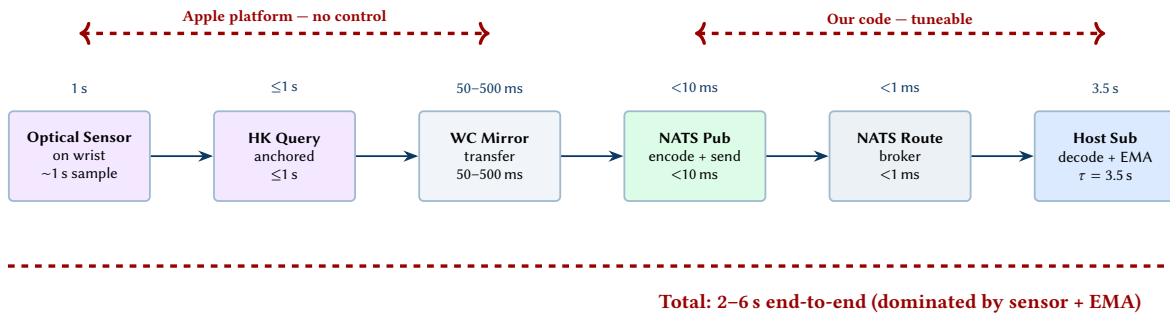


Figure 7. End-to-end heart rate latency budget. The total delay from cardiac event to displayed BPM is 2–6 seconds. The first half (sensor sampling, HealthKit query, WatchConnectivity transfer) is controlled by Apple’s platform; the second half (NATS routing, EMA smoothing) is tuneable. The EMA time constant dominates the controlled portion.

The BLE telemetry path is significantly shorter: the trainer broadcasts Indoor Bike Data every ~250 ms, and the Host decodes it immediately. The smoothing buffer adds at most 750 ms (3 samples). Power and cadence are therefore displayed within ~1 s of the pedal stroke—fast enough for the rider to perceive real-time feedback.

The agent command path is the most latency-sensitive. When the agent publishes a grade change, it must reach the BLE Control Point within 500 ms to feel responsive. The path is: Agent → NATS (<1 ms) → Host decode (<5 ms) → AgentRideCoordinator throttle (up to 500 ms) → BLE write (20–50 ms). The throttle is the bottleneck by design: it coalesces rapid grade changes to avoid flooding the trainer’s Control Point.

Insight:
The EMA τ of 3.5 s is the single largest source of controllable latency. Reducing it to 1.5 s would halve the display delay at the cost of visible jitter. This is a user-facing tradeoff that should eventually be configurable.

11 The Control Design Space

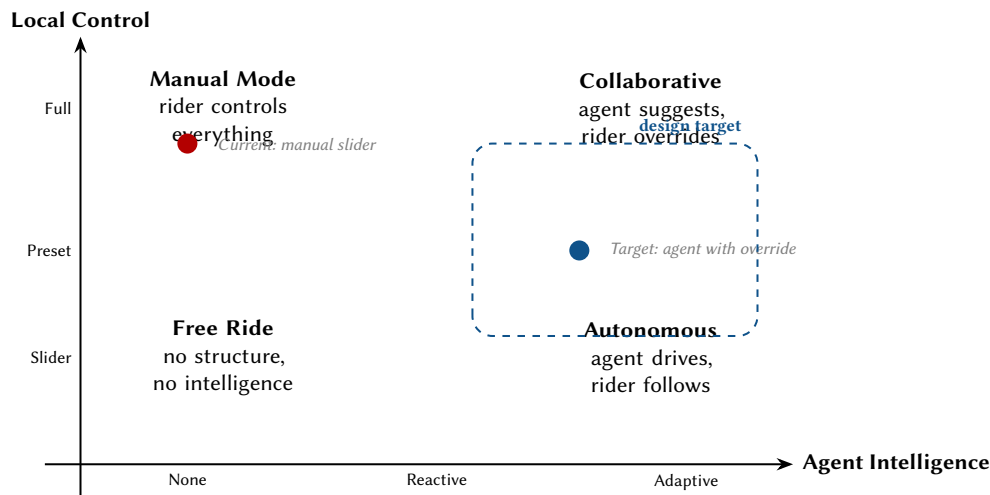


Figure 8. Orthogonal axes: Agent Intelligence vs. Local Control. These two properties are independent; a system can have high local control with no agent (manual mode) or high agent intelligence with no local override (autonomous mode). The current system sits in the upper-left quadrant (manual slider, no real agent). The design target is the centre-right: an adaptive agent with rider override capability. The dashed region marks the design target zone.

This diagram clarifies what Cycle Agent is *not*: it is not building towards a fully autonomous system where the rider has no control. The Agent–Manual Exclusion axiom (section 14) is a temporary simplification. The target architecture allows the rider to override the agent at any time—grabbing the slider should immediately suppress agent commands, and releasing it should resume agent control. This requires a priority arbitration layer that does not yet exist.

12 Failure Modes

The failure mode table reveals a pattern: every failure currently results in a frozen display with no user feedback. The consistent fix is a **staleness timer** on every data source. If no update arrives within a source-specific

Insight:
The most dangerous failure is the silent one: the agent crashes and the trainer holds the last commanded grade indefinitely. If that grade was 12%, the rider is stuck at a steep climb with no way to override it without restarting the app.

Table 1. Failure mode matrix. Each row describes a link failure, its observable effect, and the current and desired recovery behaviour.

Failed Link	What the Rider Sees	Current Behaviour	Desired Behaviour
BLE to trainer	Power, cadence, speed freeze at last value. Resistance slider stops responding.	State transitions to Disconnected. No automatic recovery. Rider must manually reconnect.	Auto-reconnect with backoff. Agent notified of hardware loss. Stale metrics greyed out after 3 s.
NATS broker down	Agent telemetry stops. No new grade targets. Manual slider still works (BLE is independent).	NATSManager enters error state. No reconnection attempt.	Exponential backoff reconnect. Local agent fallback activates cached workout. Banner: "Agent offline."
Watch disconnects	Heart rate display freezes. Zone and trend indicators stale.	No detection. Frozen value displayed indefinitely.	Staleness timer: after 10 s without a sample, display "-" and grey the zone indicator.
Bridge crashes	Heart rate relay stops. Ride lifecycle commands undelivered. Watch continues recording independently.	No detection on Host side. Watch workout continues.	Bridge heartbeat on cycle.health.v1.status. Host detects absence after 5 s and shows warning.
Agent crashes	Grade targets stop arriving. Last grade remains active on trainer.	AgentRideCoordinator holds last grade. Manual slider remains locked if agent was active.	Telemetry timeout after 5 s: unlock manual slider, notify rider "Agent disconnected", hold current grade.

window (3 s for BLE, 10 s for HR, 5 s for agent telemetry), the display should visually degrade: grey out the value, show "-", or display a warning banner.

: Staleness Visibility

Every displayed metric must have a maximum staleness threshold. When the threshold is exceeded, the metric must be visually degraded (greyed, dashed, or replaced with a placeholder) rather than displayed as if current. No stale value may appear "live."

13 Offline Preparation Opportunities

Several components can be designed or built without requiring the full device stack. These represent work that could be completed ahead of the remaining implementation phases.

13.1 Workout Schema and Library

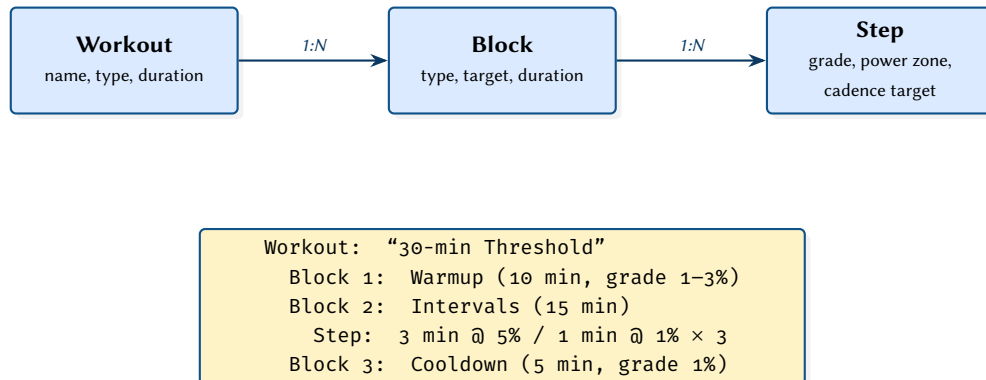


Figure 9. Proposed workout schema. A workout is a sequence of blocks; each block is a sequence of steps. This three-level hierarchy can represent intervals, pyramids, ramps, and steady-state efforts. The schema can be defined and a library of workouts authored entirely offline.

A workout schema and a library of standard workouts (sweet spot, VO₂ intervals, endurance, recovery) can be designed as JSON documents conforming to a versioned schema. These documents would be consumed by the agent at runtime. No hardware, no NATS broker, no Apple device required.

13.2 Agent Decision Engine

The agent’s decision logic—when to advance to the next interval, when to extend recovery based on heart rate, when to abort a session—can be implemented and tested as a pure function:

$$f(\text{workout plan, current metrics, elapsed time}) \rightarrow \text{next command}$$

This function has no side effects, no NATS dependency, and no BLE dependency. It can be developed in Python, Rust, or Swift and tested with recorded metric traces.

13.3 Terrain Themes and Biomes

The current terrain generator uses a single noise profile. Different workout types could map to different visual environments: flat roads for endurance sessions, mountain passes for climbing intervals, rolling hills for sweet spot work. The noise parameters, colour palettes, and skybox configurations can all be designed offline and stored as theme files.

13.4 Ride Recording and Export

A ride recording format (timestamps + metrics + agent commands) can be designed now. This enables:

- **Replay:** Re-watch a completed ride with the same terrain and metrics.
- **Export:** Produce FIT or TCX files for upload to Strava, TrainingPeaks, or other platforms.
- **Agent training:** Use recorded rides as training data for adaptive agents.

13.5 Protocol Conformance Test Suite

A test harness that connects to NATS, sends malformed envelopes, oversized payloads, rapid-fire commands, and out-of-order schema versions would validate the robustness of the protocol layer. This can run entirely on macOS without any Apple hardware.

13.6 Onboarding and Profile UI Prototypes

SwiftUI previews work without a device. The onboarding flow (enter max HR, FTP, preferred zones), the workout selection screen, and the post-ride summary can all be prototyped in Xcode previews and iterated on without hardware.

14 Architectural Invariants

The following invariants are maintained by construction across the system. Violating any of them would indicate an architectural regression.

Axiom 1 (Single Writer per BLE Characteristic). *At most one Swift Task may have a pending write to any given BLE characteristic at any time. Concurrent writes are serialised via a CheckedContinuation guard.*

Axiom 2 (NATS Lifecycle Coherence). *The NATS connection is suspended when the app enters the background and resumed when it returns to the foreground. No NATS operations are attempted during the suspended state.*

Axiom 3 (Agent–Manual Exclusion). *When an AI agent is actively publishing grade targets, the manual resistance slider is disabled. Manual and agent control never overlap.*

Axiom 4 (Deterministic Terrain). *Given the same scene seed and chunk index, the terrain generator produces byte-identical geometry. This is enforced by the absence of any non-deterministic input (no random(), no Date(), no device state) in the generation path.*

15 Future Directions

15.1 Multi-Rider Sessions

The NATS subject namespace already supports multiple agents via the agentId segment. Extending this to multiple riders—each with their own telemetry subject, their own agent, and their own terrain—would enable group training sessions where riders share the same broker but see independent workout plans.

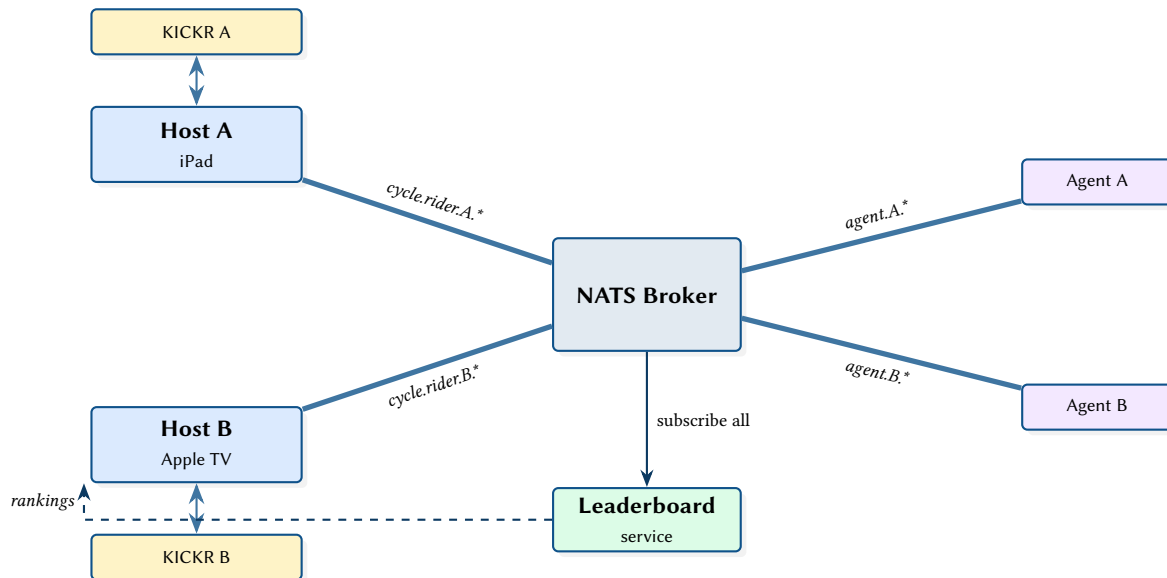


Figure 10. Multi-rider topology. Each rider has an independent Host, trainer, and agent. A leaderboard service subscribes to all rider subjects via wildcard (`cycle.rider.>`) and publishes rankings. No rider needs to know about any other rider’s infrastructure.

15.2 Adaptive Terrain from Agent State

Currently, terrain generation is independent of the workout. A natural extension is to feed the agent’s target grade into the terrain generator, producing visual hills that correspond to the resistance the rider actually feels. When the agent prescribes a 5% grade, the terrain ahead should visually rise. This closes the sensory loop between effort and environment.

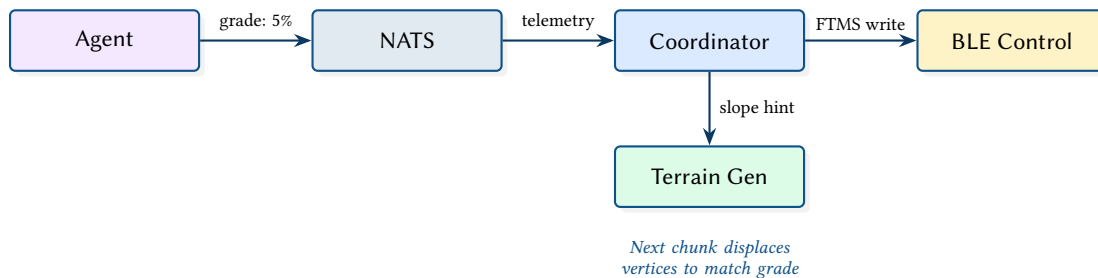


Figure 11. Adaptive terrain pipeline. The Coordinator receives the agent’s grade target and feeds it to both the BLE layer (resistance) and the terrain generator (visual slope). The rider feels and sees the same gradient simultaneously.

15.3 Voice Coaching via Agent Telemetry

The agent telemetry envelope includes a note field (a freeform string). An agent could publish coaching cues (“increase cadence”, “steady effort”, “recovery in 30 seconds”) that the Host renders as text overlays or synthesises as speech via AVSpeechSynthesizer.

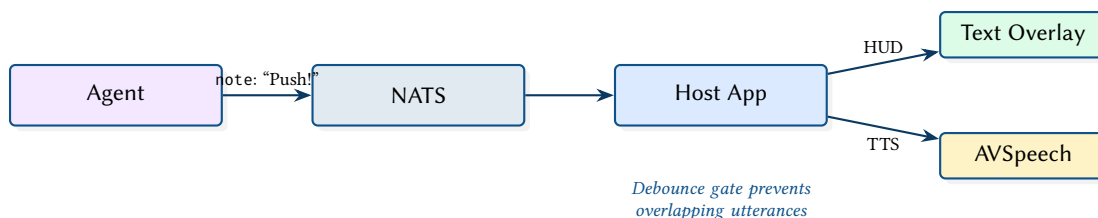


Figure 12. Voice coaching path. The agent publishes a coaching cue in the note field of its telemetry envelope. The Host renders it as a HUD text overlay and optionally synthesises it as speech. A debounce gate prevents rapid-fire cues from producing overlapping audio.

15.4 Recorded Ride Ghosts

With a ride recording format, the system could render a “ghost rider”—a translucent avatar following the path of a previous ride. This provides pacing feedback: if you are ahead of your ghost, you are riding faster than last time.

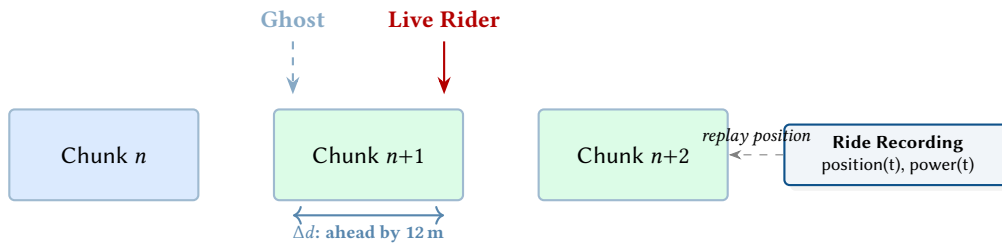


Figure 13. Ghost rider overlay. The live rider and the ghost share the same terrain. The ghost’s position is replayed from a recorded ride file. The distance delta (Δd) provides real-time pacing feedback.

15.5 ERG Mode Support

The current resistance model uses simulation mode (grade + physics coefficients). FTMS also supports ERG mode, where the trainer maintains a fixed power target regardless of cadence. Adding ERG support would enable structured power-based workouts (e.g., “hold 200 W for 5 minutes”) that are standard in platforms like Zwift and TrainerRoad.

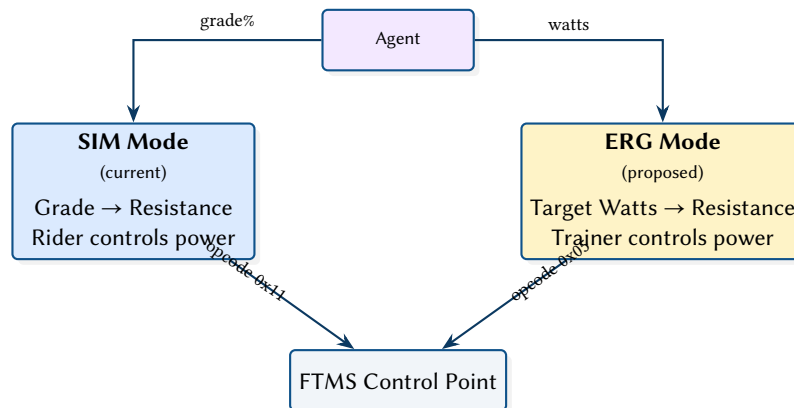


Figure 14. SIM vs. ERG mode. Both use the FTMS Control Point but with different opcodes and semantics. The agent decides which mode to use based on the workout type: SIM for hill simulation, ERG for structured power targets.

15.6 Local Agent Fallback

An embedded agent that executes a cached workout plan without NATS would provide resilience against broker unavailability. The embedded agent would implement the same protocol interface but communicate via in-process function calls rather than network messages.

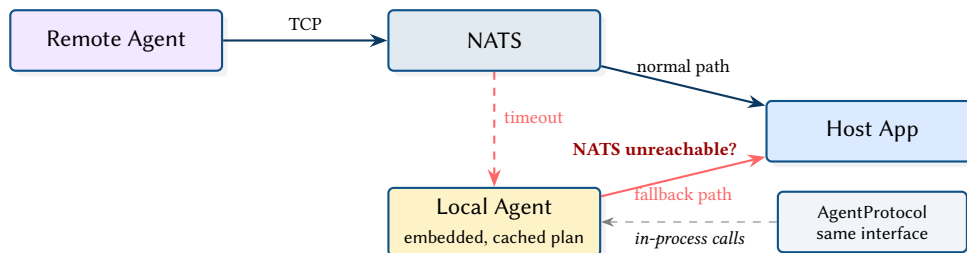


Figure 15. Local agent fallback. When the NATS broker is unreachable, the Host activates an embedded agent that executes a cached workout plan. The local agent implements the same AgentProtocol interface, so the Coordinator and BLE layer are unaware of the switch.

16 Product and Feature Ideas

The following ideas extend beyond the current v0.1 scope but are architecturally enabled by the existing decomposition.

16.1 Workout Builder UI

A drag-and-drop workout builder on iPad, where the rider constructs interval blocks visually (duration on the x-axis, intensity on the y-axis), then exports the result as a workout JSON document for the agent. This is a natural companion to the workout schema (fig. 9). The builder would produce the same versioned JSON that the agent consumes, so a rider-authored workout and an AI-generated workout are indistinguishable at the protocol level.

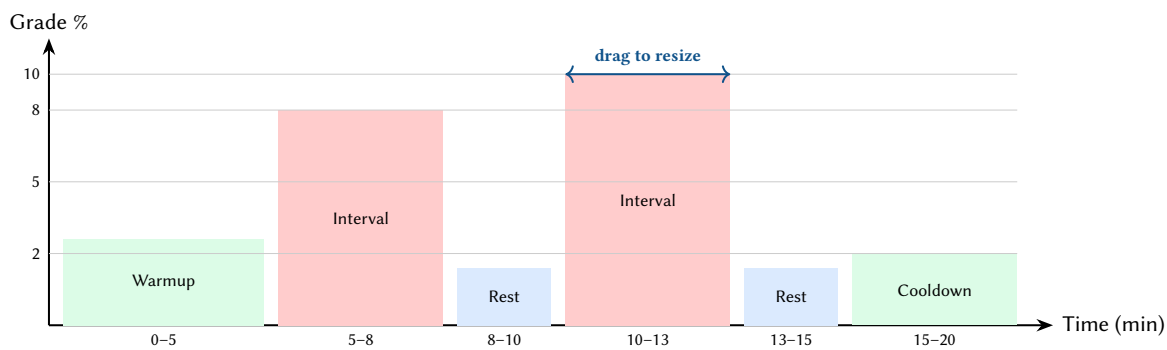


Figure 16. Workout Builder mockup. The rider drags blocks onto a time–intensity canvas. Each block maps directly to a Block/Step pair in the workout schema. The exported JSON is protocol-identical to an agent-generated workout.

16.2 Training Plan Progression

A multi-week training plan where each session builds on the last. The agent would track cumulative training load (TSS or similar metric), prescribe recovery days, and progressively overload threshold efforts. This requires a persistence layer (ride history + metrics) that does not yet exist. The data model is straightforward: a time-series of completed workout summaries, each carrying average power, normalised power, duration, and heart rate statistics.

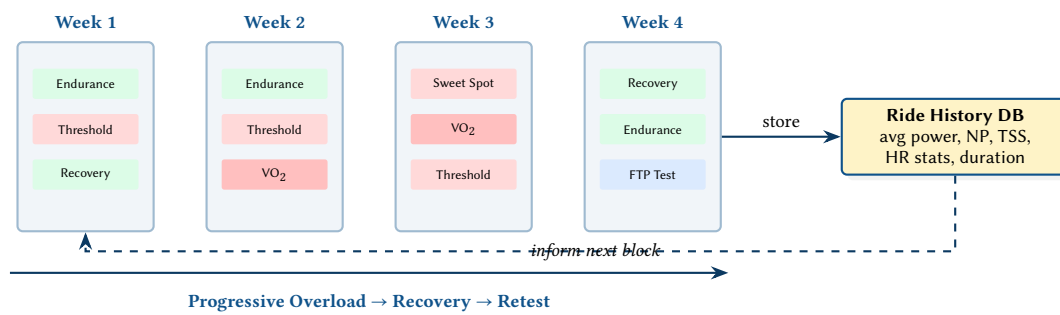


Figure 17. Training plan progression. A 4-week mesocycle with progressive overload and a recovery week. The ride history database stores session summaries; the agent uses accumulated training load to select the next session’s intensity. After the recovery week, an FTP retest calibrates the next cycle.

16.3 Live Metrics Dashboard via NATS

Because all telemetry flows through NATS, any subscriber can build a dashboard. A web application subscribing to `cycle.rider.*.telemetry` and `cycle.health.v1.hr` could display a real-time training dashboard on a laptop, a wall-mounted monitor, or a coach’s tablet. The implementation is a NATS WebSocket bridge plus a lightweight frontend. No changes to the Host or Bridge are required.

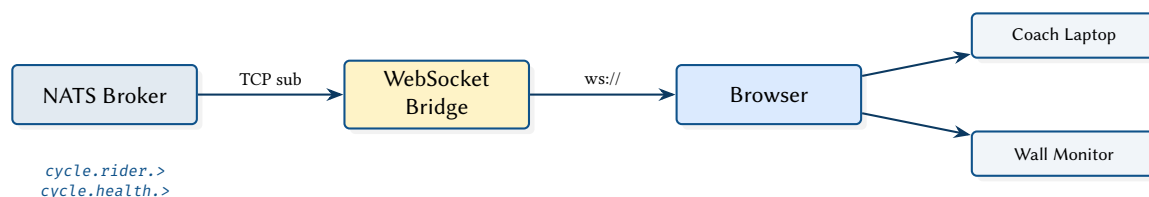


Figure 18. Live dashboard path. NATS natively supports WebSocket listeners. A browser subscribes via the WebSocket bridge and renders live metrics. Multiple display surfaces can subscribe simultaneously—no changes to the Host, Bridge, or Agent required.

16.4 Social Riding and Leaderboards

Multiple riders on separate KICKR trainers, each with their own Host and Watch stack, could share a NATS broker. Each rider publishes their metrics on a rider-specific subject (`cycle.rider.{riderId}.metrics`). A leaderboard service subscribes to all riders and publishes rankings. Each Host subscribes to the leaderboard and overlays positions on the terrain. The NATS subject namespace supports this without modification.

16.5 Post-Ride Analytics Report

After a ride, generate a PDF or in-app report containing: power curve, heart rate trace, zone time distribution, interval compliance (did the rider hit the target power for each interval?), and a comparison with the previous session. The data sources are the ride recording and the workout plan. The report generation can happen

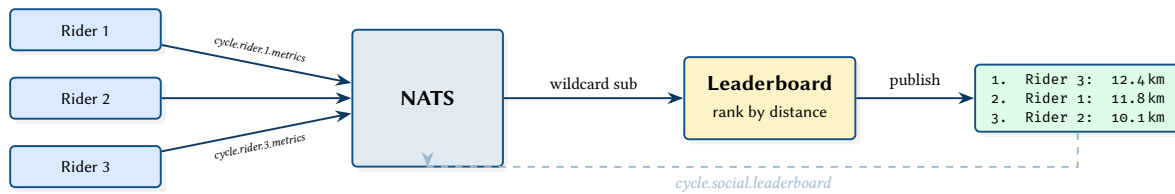


Figure 19. Social leaderboard architecture. Each rider publishes metrics to a rider-specific subject. The leaderboard service subscribes via wildcard, computes rankings, and publishes them on a shared subject. Each Host subscribes to the leaderboard and overlays positions on the HUD.

entirely on-device or be delegated to the agent (which could use an LLM to produce natural-language coaching feedback).

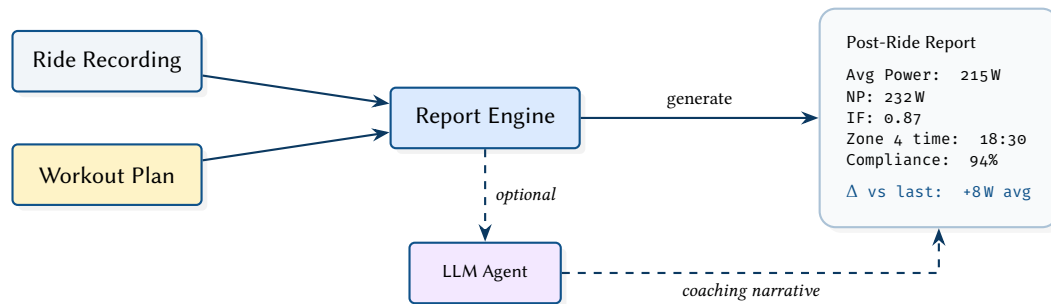


Figure 20. Post-ride analytics pipeline. The report engine compares the ride recording against the workout plan to compute compliance metrics. An optional LLM agent adds a natural-language coaching summary.

16.6 Zwift/TrainerRoad Course Import

The workout schema is generic enough to represent courses from other platforms. A converter that reads ZWO (Zwift workout format) or MRC/ERG (TrainerRoad) files and produces Cycle Agent workout JSON would let riders bring their existing training libraries. This is a standalone CLI tool with no runtime dependency.

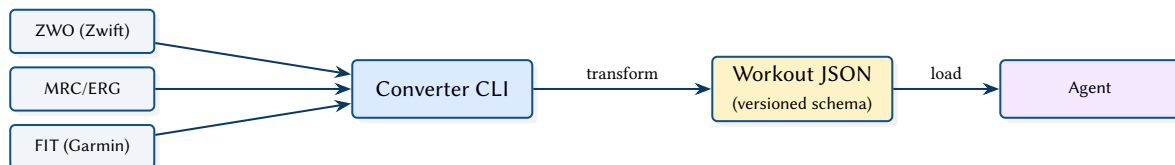


Figure 21. Course import pipeline. The converter CLI is a standalone tool that transforms third-party workout formats into the Cycle Agent versioned JSON schema. The output is indistinguishable from a natively authored workout.

16.7 Music Sync Agent

An agent that subscribes to the rider's current power output and selects music with a matching tempo. When power rises (sprint), the agent selects high-BPM tracks; during recovery, it selects lower-tempo tracks. This agent would publish on a `cycle.agent.music.telemetry` subject with the current track name and target BPM range. A companion app (or Shortcuts automation) would control Apple Music playback accordingly.

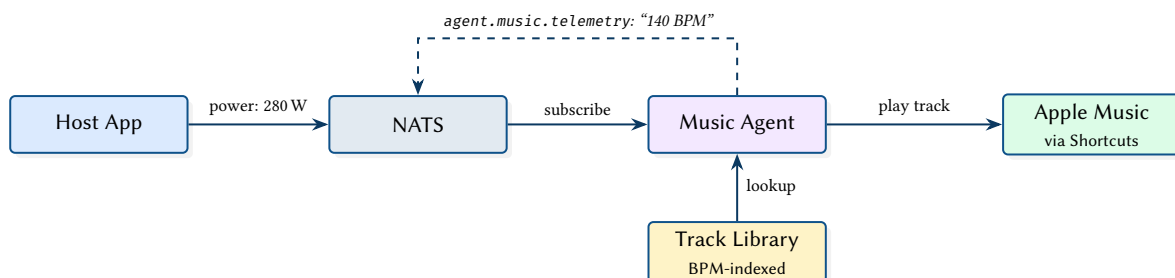


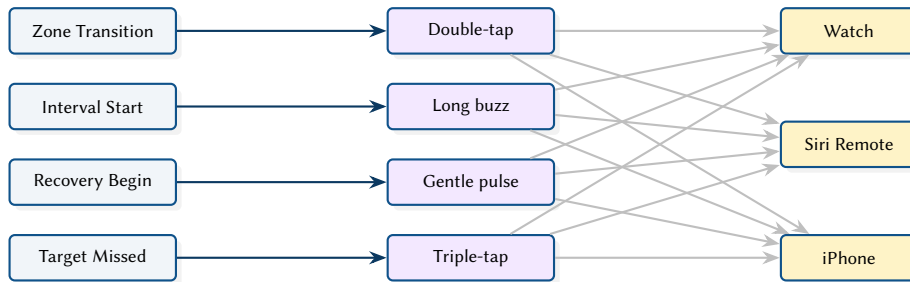
Figure 22. Music sync architecture. The music agent maps instantaneous power to a target BPM range, selects a track from a BPM-indexed library, and triggers playback via Shortcuts or MusicKit. It publishes its state as standard agent telemetry.

17 Experiential and Sensory Ideas

These ideas exploit the multi-sensory potential of the Apple device ecosystem to make indoor cycling feel less like staring at a screen.

17.1 Haptic Feedback on Zone Transitions

The Apple TV Siri Remote, iPhone, and Apple Watch all contain haptic engines. When the rider crosses a heart rate zone boundary or an interval starts, a distinct haptic pattern communicates the event without requiring the rider to glance at the screen. The Watch is the most useful target: it is already on the rider's wrist and can deliver wrist taps during a workout session. Zone entry could use a double-tap, interval start a long buzz, and recovery a gentle pulse.



Insight:
Haptic cues reduce visual load. A rider in a high-intensity interval should not need to read a screen to know they have entered Zone 5.

Figure 23. Haptic event mapping. Each training event maps to a distinct haptic pattern. All three haptic-capable devices receive the same event; the rider feels the transition on whichever device is closest (typically the Watch on the wrist).

17.2 Time-of-Day Lighting Cycle

The SceneKit environment currently uses static lighting. A natural extension is to map workout phase to time of day: warm golden light during the warmup (dawn), bright overhead light during intervals (noon), and orange-red light during the cooldown (sunset). The lighting change is gradual enough to be subliminal but provides an ambient sense of workout progression. The implementation is a single `SCNLight.color` animation driven by elapsed workout percentage.

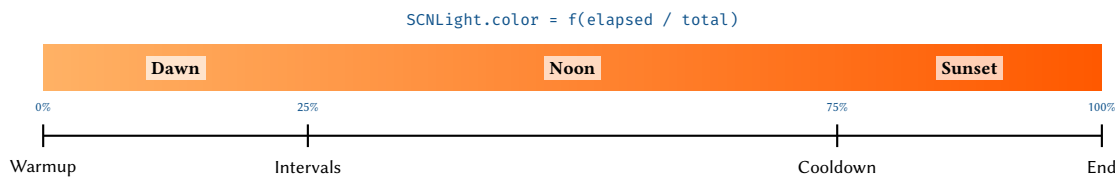


Figure 24. Time-of-day lighting mapped to workout progress. The colour temperature transitions from warm (dawn) through neutral (noon) to warm again (sunset) as a continuous function of elapsed workout percentage. The rider perceives ambient progression without reading a clock.

17.3 GPX Route Replay

Instead of infinite procedural terrain, load a real-world GPS route (GPX file) and generate terrain matching the elevation profile. The rider sees the actual route they would ride outdoors: the exact climbs, descents, and flats. The agent sets grade to match the route's gradient at the rider's virtual position. This converts Cycle Agent into a route-previewer for outdoor rides.

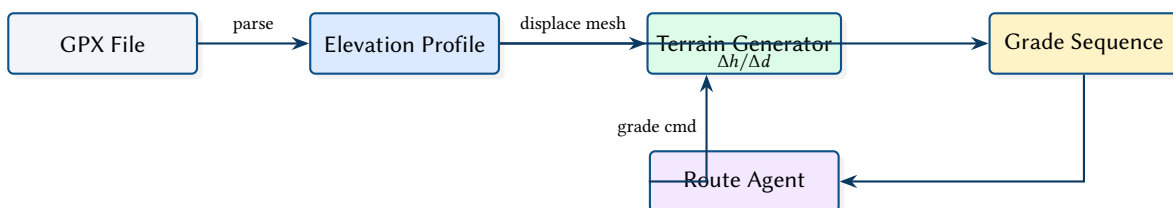


Figure 25. GPX route replay pipeline. The elevation profile drives both the terrain mesh and the agent's grade sequence. The terrain generator displaces its mesh to match real-world elevation; the route agent publishes grade commands that match the gradient at the rider's current virtual position.

17.4 Metal Shader Effects

SceneKit supports custom Metal fragment shaders via `SCNProgram`. Visual intensity effects could communicate effort without numbers:

- **Speed lines:** Radial blur streaks at high power, absent at low power. Communicates velocity viscerally.
- **Zone-coloured vignette:** Screen edges tinted to the current heart rate zone colour (blue for Zone 1, green for Zone 2, through to red for Zone 5). The centre of the screen stays clear; the peripheral tint is subliminal.

- **Heat haze:** A subtle screen distortion at sustained high power (>300 W for >30 s). Communicates accumulated effort without text.
- **Desaturation on fatigue:** If heart rate drift is detected (HR rising at constant power), gradually desaturate the terrain colours. The world literally loses colour as the rider fatigues.

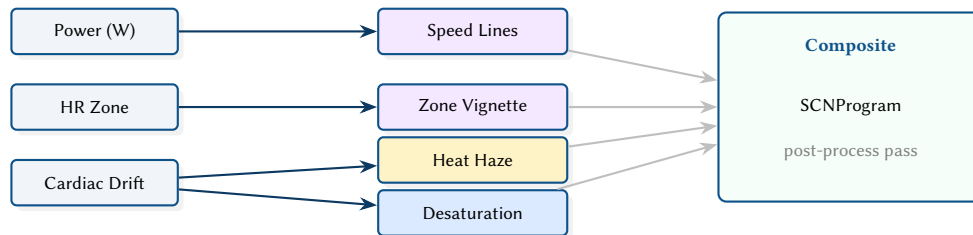


Figure 26. Metal shader pipeline. Each physiological signal drives an independent visual effect. The effects are composited in a single post-processing pass via SCNProgram. All effects are additive: a rider in Zone 5 at 300 W with cardiac drift sees speed lines, a red vignette, heat haze, and desaturated terrain simultaneously.

17.5 Siri and Shortcuts Integration

The Host App could register Siri Intents and Shortcuts actions: “Hey Siri, start a 30-minute threshold workout on Cycle Agent.” This would launch the app, connect to the trainer, publish a ride, start to the Bridge, and instruct the agent to load the named workout—all without touching the screen. The Shortcuts action could also accept parameters: duration, workout type, target intensity.

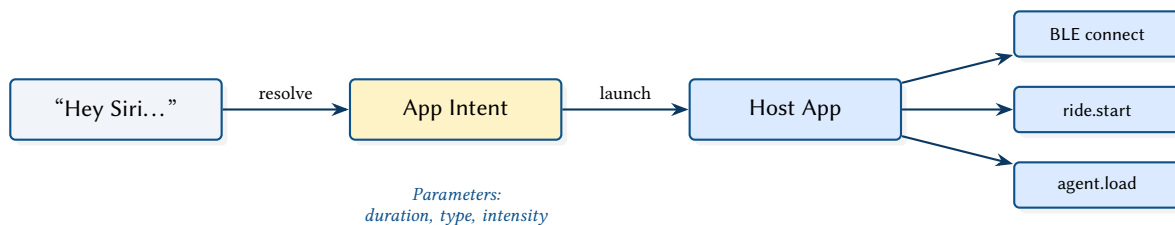


Figure 27. Siri integration sequence. A single voice command resolves an App Intent, launches the Host, and triggers a cascade of actions: trainer connection, ride start, and workout load. The rider goes from voice command to pedalling without touching the screen.

17.6 SharePlay Group Rides

Apple’s SharePlay framework enables synchronised experiences over FaceTime. Two riders on FaceTime could share a workout session: both see the same terrain, the same interval timer, and each other’s power output overlaid on the HUD. The synchronisation primitive is SharePlay’s GroupActivity API, which provides a shared state channel. The NATS broker handles the actual metric exchange; SharePlay handles the session coordination and voice channel.

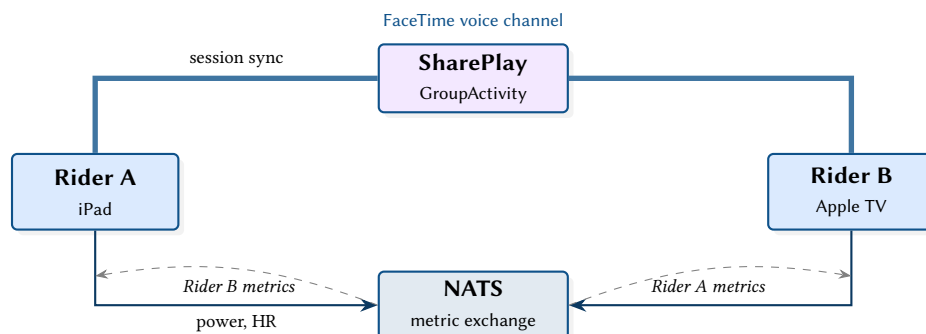


Figure 28. SharePlay group ride architecture. SharePlay manages session coordination and the FaceTime voice channel. NATS handles the actual metric exchange. Each rider’s Host subscribes to the other rider’s metrics and overlays them on the HUD.

18 Data and Intelligence Ideas

18.1 Critical Power Model

From accumulated ride data, the system can build a power-duration curve and estimate the rider’s critical power (CP) and anaerobic work capacity (W'). The model is:

$$P(t) = CP + \frac{W'}{t}$$

With enough data points (rides containing max efforts at various durations), a least-squares fit produces CP and W' estimates. These values feed directly into the agent's decision engine: the agent knows how long the rider can sustain a given power and can prescribe intervals that target specific energy systems.

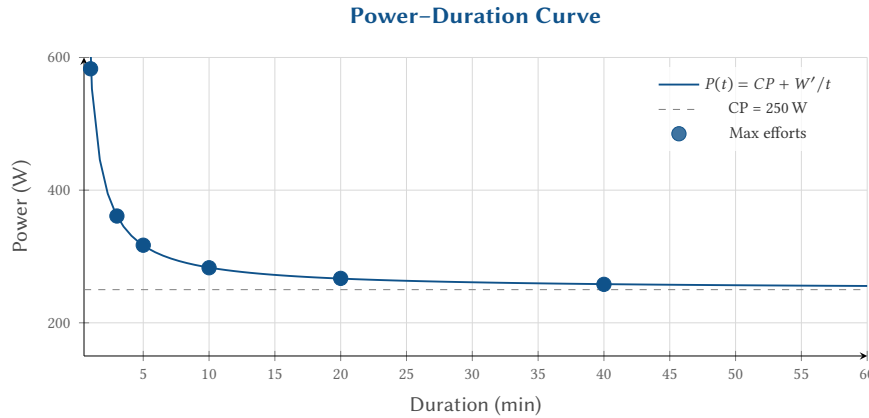
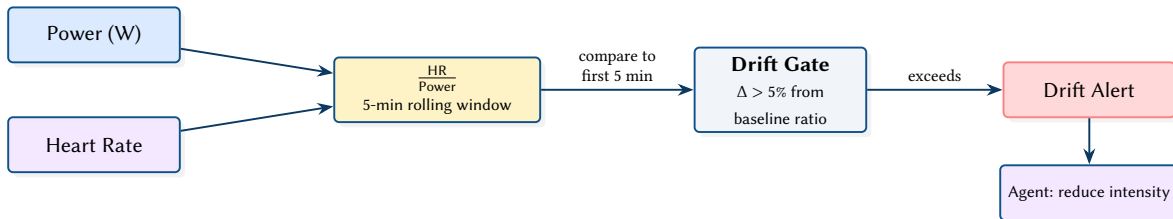


Figure 29. Power-duration curve with CP model fit. The hyperbolic curve $P(t) = CP + W'/t$ is fitted to the rider's best efforts at various durations. CP (the asymptote) represents sustainable aerobic power; W' (the area above CP) represents the finite anaerobic reserve.

18.2 Cardiac Drift Detection

Cardiac drift—heart rate creeping upward at constant power—is a key signal in endurance training. It indicates progressive dehydration, thermal stress, or accumulating fatigue. The system already has both power (from BLE) and heart rate (from Watch). A drift detector that monitors the ratio HR/Power over a rolling 5-minute window and fires a “drift detected” event when the ratio exceeds a threshold would give the agent a new input signal. The agent could respond by reducing intensity, extending the next recovery period, or ending the session.



Insight: Cardiac drift detection is the simplest form of physiological intelligence; it requires only two signals the system already has and a sliding window ratio. No machine learning required.

Figure 30. Cardiac drift detector. The HR/Power ratio is computed over a rolling 5-minute window and compared to the baseline ratio from the first 5 minutes. When the ratio drifts above a 5% threshold, the agent is notified and can reduce intensity or extend recovery.

18.3 Perceived Exertion Prediction

Given enough ride history with RPE (Rating of Perceived Exertion) labels—which the rider could provide via a post-ride prompt on the Watch—a lightweight model could predict RPE from the current session's power, heart rate, cadence, and duration. The predicted RPE gives the agent a real-time “how hard does this feel?” signal without interrupting the rider. A simple model: linear regression on (IF, HR drift, duration, cadence variance) → RPE where IF is Intensity Factor (NP/FTP).

18.4 Digital Twin / Simulation Mode

A virtual rider that generates synthetic BLE telemetry based on a configurable rider profile (FTP, max HR, cadence preference, fatigue curve). The digital twin connects to NATS and publishes metrics as if it were a real trainer. This enables:

- Agent development without any hardware.
- Automated testing of failure modes (simulate a BLE drop mid-interval).
- Demonstration mode for presentations.
- Regression testing: replay a recorded ride and verify the agent produces the same grade sequence.

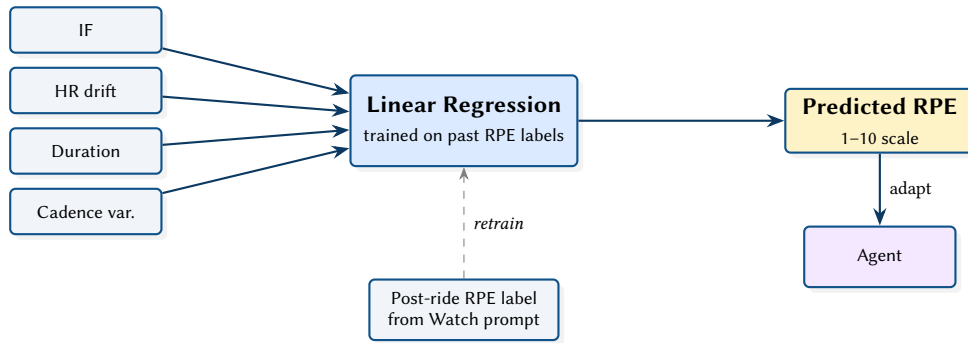


Figure 31. RPE prediction model. Four features derived from the current session feed a lightweight linear regression trained on the rider’s historical RPE labels. The predicted RPE gives the agent a real-time fatigue estimate. The model retrains incrementally after each ride.

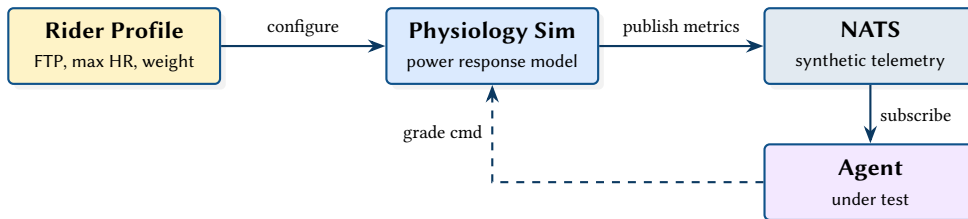


Figure 32. Digital twin feedback loop. The physiology simulator generates metrics from a rider profile, publishes them to NATS, and the agent responds with grade commands that feed back into the simulator. This closed loop runs without any Apple hardware or BLE connection.

18.5 Third-Party Agent Marketplace

Because agents are standalone NATS clients with a published protocol, third parties could build and distribute agents independently:

- A **race simulation agent** that mimics the pacing of a real road race (attacks, breakaways, group dynamics).
- A **recovery agent** that monitors heart rate variability and caps intensity to stay aerobic.
- A **test protocol agent** that runs standardised FTP tests (20-minute test, ramp test) and computes the result.
- A **LLM coaching agent** that uses a large language model to provide conversational coaching during the ride, adapting to the rider’s responses and perceived effort.

The marketplace is a directory of agent configurations (NATS connection details + agent ID + workout library). No App Store review required; agents are server-side processes that the rider connects to by entering a NATS URL and agent ID.

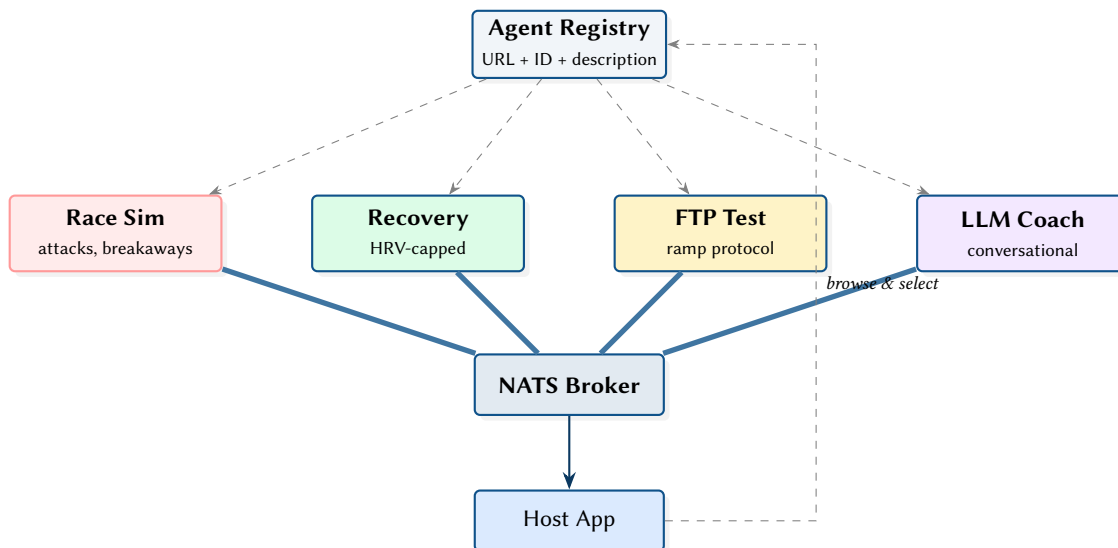


Figure 33. Third-party agent marketplace. Each agent is an independent NATS client. The registry is a simple directory (JSON or web API) listing available agents. The rider browses the registry from the Host App, selects an agent, and the Host connects to the corresponding NATS subject namespace. No App Store distribution required.

18.6 Observability via NATS Subjects

Every component could publish diagnostic events to a structured subject hierarchy:

- `cycle.diag.host.ble.*` — BLE connection events, write latencies, CCCD state changes.
- `cycle.diag.host.render.*` — Frame times, chunk generation durations, camera positions.
- `cycle.diag.bridge.wc.*` — WatchConnectivity transfer latencies, message drop counts.
- `cycle.diag.agent.*` — Decision latencies, state transitions, error counts.

A diagnostic subscriber (command-line tool or web dashboard) would subscribe to `cycle.diag.>` (NATS wildcard) and display a real-time operational view of the entire system. This is the same pattern used in production microservice architectures and costs nothing to add—the overhead of publishing a small JSON payload to a NATS subject that has no subscribers is effectively zero.

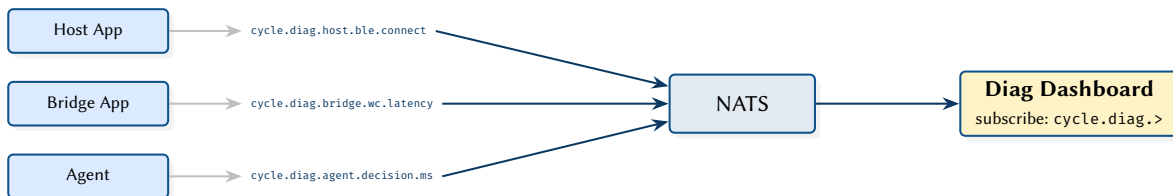


Figure 34. Observability via NATS subjects. Each component publishes diagnostic events to a structured subject hierarchy. A wildcard subscriber (`cycle.diag.>`) captures all diagnostics. When no subscriber is connected, the publish cost is effectively zero.

18.7 Standalone Power Meter Support

Not every rider has a smart trainer. Many cyclists own pedal-based (Garmin Rally, Favero Assioma) or crank-based (Stages, 4iiii) power meters that broadcast via BLE Cycling Power Service (CPS). The Host could support a second BLE connection mode: pair with a CPS power meter for metrics display, while using a “dumb” trainer (or no trainer at all, for rollers). In this mode, the agent cannot control resistance (no FTMS Control Point), but it can still provide coaching, zone-based guidance, and post-ride analytics based on the power data.

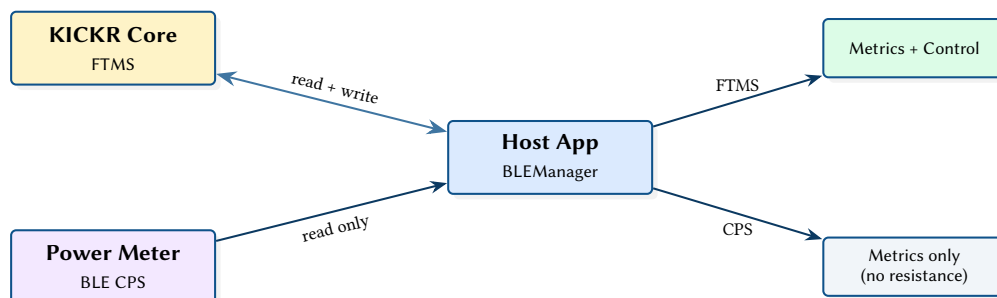


Figure 35. Dual BLE service support. The Host auto-detects whether the paired device exposes FTMS (smart trainer) or CPS (standalone power meter). FTMS enables full metrics and resistance control; CPS provides metrics only. The agent adapts: with CPS, it provides coaching cues instead of grade commands.

18.8 ANT+ Bridge

Older trainers and power meters use ANT+ rather than BLE. A small bridge process running on a machine with a USB ANT+ dongle (e.g., a Raspberry Pi) could translate ANT+ FE-C (Fitness Equipment Control) messages into NATS publishes, making legacy hardware accessible to the Cycle Agent ecosystem. The bridge would be a standalone NATS client, following the same role–device separation principle.

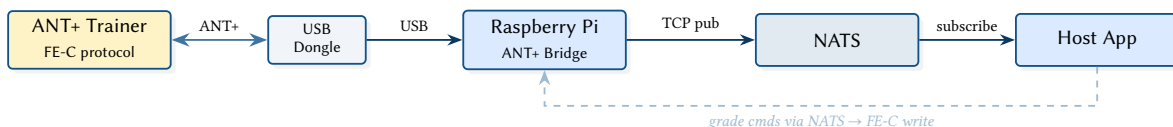


Figure 36. ANT+ bridge architecture. A Raspberry Pi with a USB ANT+ dongle translates FE-C messages to and from NATS. The Host App sees the same telemetry subjects regardless of whether the trainer uses BLE or ANT+. The bridge is a standalone NATS client following the role–device separation principle.

19 Accessibility and Inclusivity

Indoor cycling should be accessible to riders with visual, motor, and cognitive impairments. The current UI has no accessibility provisions beyond SwiftUI’s built-in VoiceOver support.

19.1 VoiceOver and Periodic Announcements

A rider with low vision cannot glance at a HUD. The system should periodically announce key metrics via VoiceOver or a dedicated audio channel: “Power 220 watts, heart rate 145, Zone 3.” The announcement cadence should be configurable (every 30 s, every interval transition, or on demand via a Siri Remote button press on Apple TV).

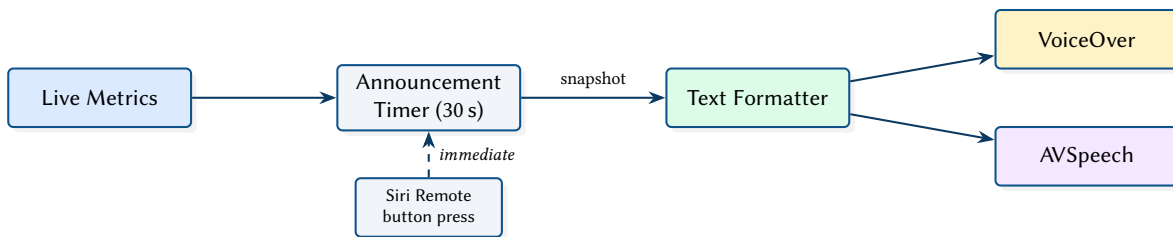


Figure 37. Accessibility announcement pipeline. Metrics are periodically formatted into spoken text and delivered via VoiceOver or direct speech synthesis. A button press on the Siri Remote triggers an immediate announcement outside the normal cadence.

19.2 Dynamic Type and High Contrast

The HUD metrics (power, cadence, speed, heart rate) must scale with the system Dynamic Type setting. A rider who sets their system font to XXL should see correspondingly larger numbers on the HUD. Similarly, the zone colour palette should have a high-contrast variant for riders with colour vision deficiency: patterns (hatching, borders) in addition to colour to distinguish zones.

19.3 Reduced Motion Mode

When the system Reduce Motion accessibility setting is enabled, the terrain should stop scrolling and the camera should remain static. The HUD continues to update metrics, but the visual environment becomes a still landscape. This prevents motion sickness for riders who are sensitive to screen movement while physically stationary.

19.4 Switch Control and External Devices

Apple TV supports Switch Control for riders with motor impairments. All interactive elements (device pairing, workout selection, resistance slider) must be reachable via the switch scanning interface. The resistance slider presents a particular challenge: it must support increment/decrement via switch taps rather than continuous drag.

20 Device Integration Ideas

20.1 Smart Fan Control

The Wahoo Headwind and similar smart fans expose a BLE GATT profile for speed control. The Host could pair with a fan as a secondary BLE peripheral and map fan speed to rider power output: low fan at recovery, maximum airflow at threshold and above. This is a single additional BLE characteristic write, piggybacking on the existing CoreBluetooth infrastructure.

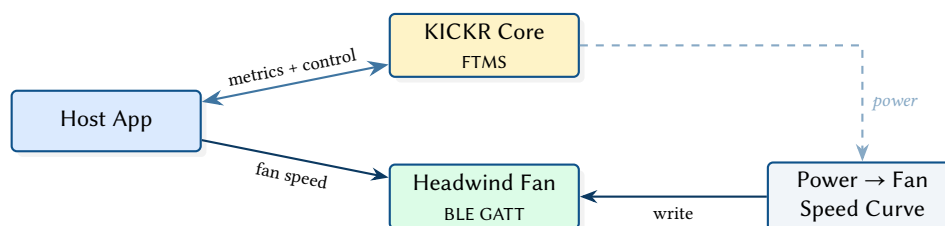


Figure 38. Smart fan integration. The Host maintains a second BLE connection to a fan peripheral. Fan speed is derived from instantaneous power via a tuneable curve. The fan write is independent of the trainer write and does not contend for the same characteristic.

20.2 External Display / Stage Manager

On iPad with Stage Manager or USB-C HDMI output, the terrain view could render on the external display while the iPad screen shows controls and detailed metrics. SceneKit supports rendering to a secondary UIScreen via UIWindowScene. This gives the rider a cinema-scale terrain view on a TV with fine-grained control on the tablet in their hands.

20.3 Picture-in-Picture on iPad

Many riders watch video content (Netflix, YouTube, race footage) while training. The Host App could support Picture-in-Picture mode, where the terrain and HUD shrink to a floating overlay while a video app occupies

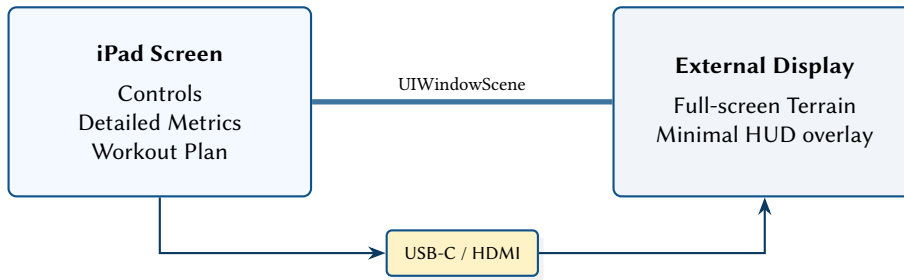


Figure 39. Dual-screen layout. The iPad renders controls and detailed data; the external display renders the immersive terrain. Both share the same RideRenderer state via a secondary UIWindowScene.

the main screen. Alternatively, the HUD alone (power, HR, cadence as large numbers) could float as a PiP overlay without the terrain, for riders who prefer video entertainment with a minimal data overlay.

20.4 Apple Watch Complications and Live Activities

A watchOS complication showing the current workout status (elapsed time, power average, HR zone) provides at-a-glance data without raising the wrist. On iPhone, an iOS Live Activity on the lock screen could display the same data for a coach or spectator standing nearby. Both use WidgetKit and require only a data source—the Bridge already has all the necessary metrics from its NATS subscription and WatchConnectivity relay.

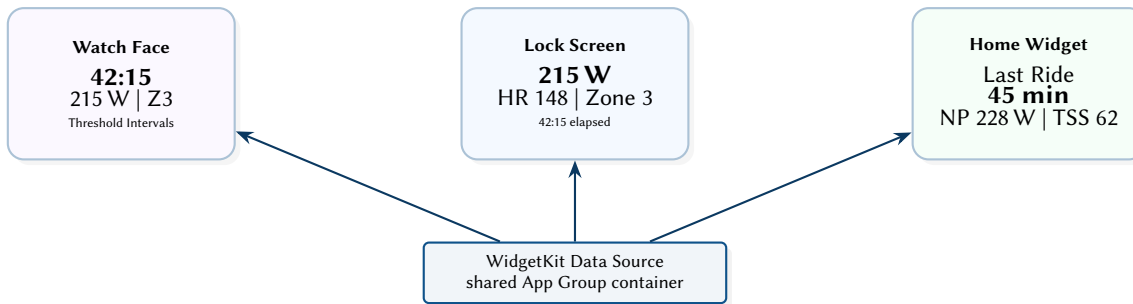


Figure 40. WidgetKit surfaces. A shared App Group container provides data to three widget types: a watchOS complication (live during ride), an iOS Live Activity (lock screen), and a home screen widget (last ride summary). All three read from the same data source.

20.5 iCloud Sync via CloudKit

Rider profile (FTP, max HR, zones), workout library, and ride history should sync across devices via CloudKit. A rider who builds a workout on their iPad should see it available on their Apple TV. A completed ride recorded on the Apple TV should appear in the iPhone’s ride history. CloudKit provides conflict-free sync for structured records with minimal server-side code.

21 Advanced Metrics and Training Science

21.1 Power Smoothing Modes

Raw instantaneous power fluctuates wildly—a single pedal stroke can spike from 50 W to 400 W and back. The current 3-sample buffer provides minimal smoothing. A configurable smoothing window (3 s, 5 s, 10 s, 30 s) would serve different use cases: 3 s for sprint training where responsiveness matters, 30 s for endurance where trend matters more than instant.

21.2 Normalised Power and Training Stress

Normalised Power (NP) captures the physiological cost of variable-power efforts. It is computed as the fourth root of the rolling 30-second average of power raised to the fourth power:

$$NP = \sqrt[4]{\frac{1}{N} \sum_{i=1}^N P_{30s,i}^4}$$

From NP, two key training metrics follow:

- **Intensity Factor (IF):** IF = NP/FTP. A value of 1.0 means the ride was at threshold.
- **Training Stress Score (TSS):** TSS = $\frac{\text{duration} \times \text{NP} \times \text{IF}}{\text{FTP} \times 3600} \times 100$. Quantifies cumulative training load.

These can be computed in real-time during the ride and displayed on the HUD. TSS is the foundation of the training plan progression system (fig. 17): the agent uses accumulated TSS to decide whether to prescribe more load or more recovery.

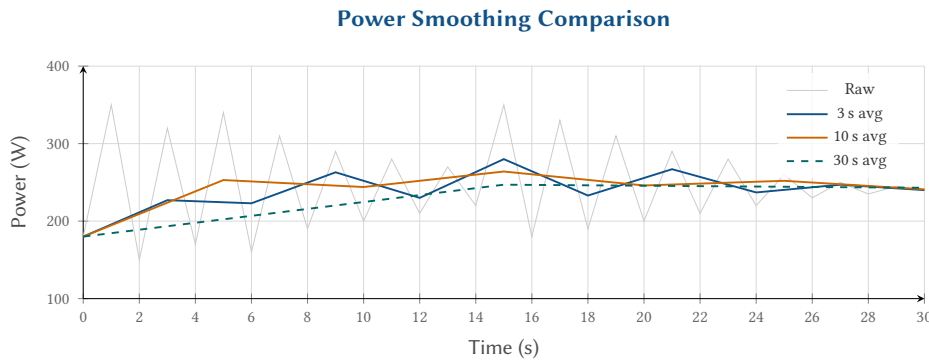


Figure 41. Power smoothing comparison. Raw power (gray) fluctuates with each pedal stroke. Increasing the smoothing window from 3 s to 30 s progressively eliminates noise at the cost of responsiveness. The rider should choose the window based on workout type: short for sprints, long for endurance.

21.3 Lap and Segment Tracking

The system should support manual laps (button press on Siri Remote or Watch) and automatic laps (at each interval boundary). Each lap records: average power, normalised power, average HR, max HR, average cadence, and duration. The post-ride report (fig. 20) would include a per-lap breakdown table.

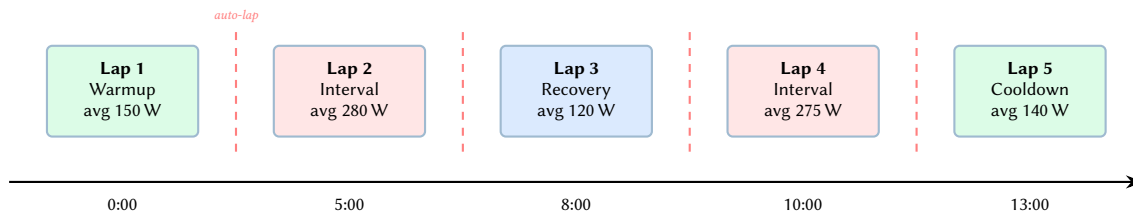


Figure 42. Lap tracking with automatic boundaries. Lap markers are inserted automatically at each interval transition (dashed lines). Each lap records per-interval metrics for the post-ride report.

21.4 Pedal Balance and Efficiency

Dual-sided power meters (Garmin Rally, Favero Assioma Duo) report left/right power balance. A persistent imbalance (e.g., 55/45) may indicate injury compensation, bike fit issues, or technique problems. The HUD could display a real-time balance bar, and the agent could flag significant deviations: “Left leg dominance increasing—check cleat position.”

21.5 Cadence-Based Coaching

The agent currently targets only grade (resistance). A richer model would also target cadence: “spin at 100 rpm” for neuromuscular drills, “grind at 60 rpm” for strength work. The agent would publish a targetCadence field in its telemetry, and the HUD would display it alongside the target grade. The trainer cannot enforce cadence (only the rider’s legs can), but the visual cue and the agent’s coaching notes provide guidance.

21.6 Readiness Score from HealthKit

Before a workout starts, the system could query HealthKit for overnight data: resting heart rate, heart rate variability (HRV), sleep duration, and sleep quality. A simple readiness score (weighted sum of z-scores against the rider’s personal baselines) would indicate whether today is a high-intensity or recovery day. The agent could automatically adjust the workout plan based on readiness.

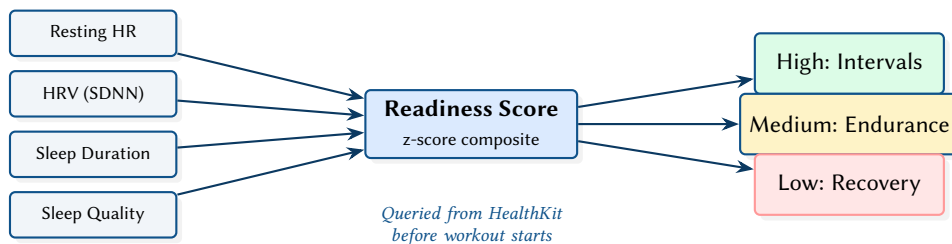


Figure 43. Readiness score from HealthKit data. Four overnight metrics are combined into a composite score that routes the rider to the appropriate workout intensity. The agent adjusts the session plan before the first pedal stroke.

21.7 Real-Time Power Curve Comparison

During a ride, the system could track the rider’s best efforts at standard durations (5 s, 1 min, 5 min, 20 min) and compare them in real-time against the all-time power curve stored in the ride history database. A HUD

overlay would show: “5-min best today: 285 W (PB: 292 W, 97%).” When the rider approaches or exceeds a personal best, the agent publishes a coaching cue and the Watch delivers a haptic alert.

22 Gamification and Engagement

22.1 Achievement System

Milestones and badges reward consistency and progression:

- **Distance:** First 100 km, 1000 km, 5000 km cumulative.
- **Consistency:** 3 rides/week for 4 weeks, 8 weeks, 12 weeks.
- **Power:** New FTP record, first 300 W sprint, first 1-hour ride above threshold.
- **Climbing:** Cumulative elevation gained (virtual metres from agent grade commands).
- **Social:** First group ride, first leaderboard top-3.

Achievements are stored locally and optionally synced via CloudKit. They have no gameplay impact—they are purely motivational.

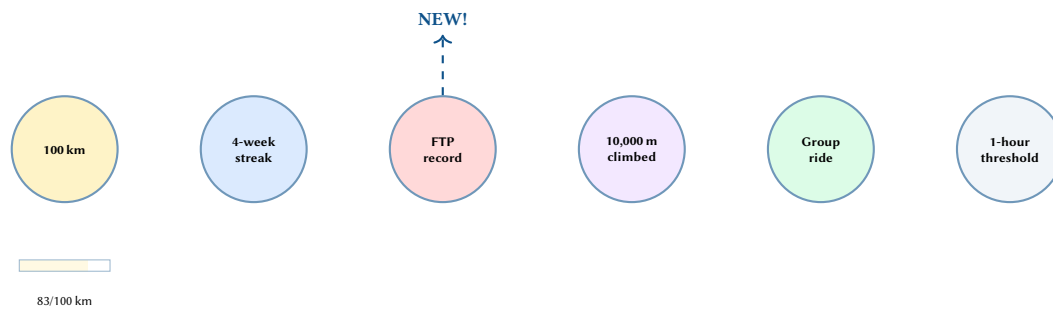


Figure 44. Achievement badges. Each badge tracks a specific milestone. Locked badges show progress bars; unlocked badges animate with a highlight. Achievements sync via CloudKit across devices.

22.2 Terrain Theme Unlocks

New terrain biomes (desert, snow, tropical, volcanic, night city) could be unlocked by completing specific achievements or accumulating virtual kilometres. This provides a concrete reward for training consistency: the world literally expands as the rider trains more. Theme files are JSON configurations (noise parameters, colour palette, skybox, ambient sound) that the terrain generator loads at ride start.

22.3 Strava Live Segment Integration

The Strava API exposes “starred segments” with effort history. During a GPX route replay (fig. 25), the system could detect when the rider enters a starred segment and display a real-time comparison against their PR or the KOM/QOM time. The terrain HUD would show the segment leaderboard position, time delta, and distance remaining. This brings Strava’s competitive element into the indoor environment.

23 Real-World Geography

The procedural terrain generator produces abstract landscapes. A natural evolution is to derive terrain from real-world geographic data, so the rider sees actual mountains, valleys, and coastlines rather than noise-generated hills. Multiple data sources make this feasible.

23.1 Elevation Data: SRTM and Open Topography

NASA’s Shuttle Radar Topography Mission (SRTM) provides global elevation data at 30-metre resolution, freely available via the USGS EarthExplorer or OpenTopography APIs. The data format is a regular grid of height values (HGT files)—exactly the input the chunk-based terrain generator already consumes. Instead of displacing vertices with simplex noise, the generator would sample the SRTM grid at the rider’s virtual position.

The critical constraint is resolution. SRTM at 30 m means each vertex in a 64×64 chunk covers approximately 2×2 km of real terrain. For a route-scale view (5–50 km), this is adequate for the visual impression of mountains and valleys. For close-up detail (individual road switchbacks), higher-resolution data sources are needed.

23.2 OpenStreetMap Road Geometry

OpenStreetMap (OSM) provides road centrelines with tags for surface type, width, and road classification. The system could:

1. Extract the road centreline from OSM for the loaded GPX route.

Insight:
The chunk interface is the key abstraction. The rest of the system—camera, speed mapping, HUD—does not know or care whether a chunk was generated from noise or from SRTM data. This is the payoff of the procedural terrain architecture: it generalises to real-world data without structural change.

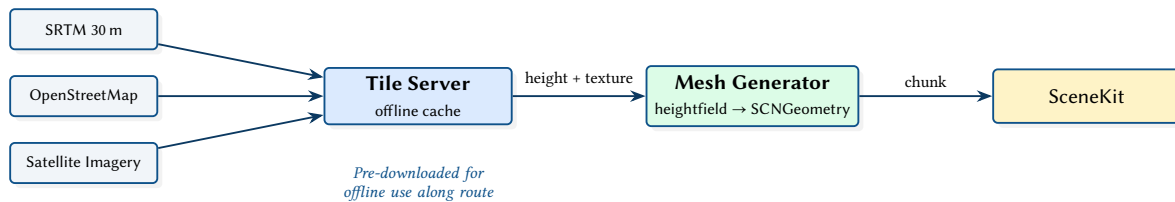


Figure 45. Real-world terrain pipeline. SRTM elevation data, OSM road vectors, and satellite imagery are fetched from public APIs, cached locally as tiles, and fed to the mesh generator. The generator produces SCNGeometry chunks identical in interface to the procedural generator.

2. Render the road as a ribbon geometry draped over the SRTM terrain mesh.
3. Use road surface tags to select texture: asphalt, gravel, cobblestone.
4. Colour-code the road by gradient (green for flat, yellow for moderate, red for steep) to preview what is coming.

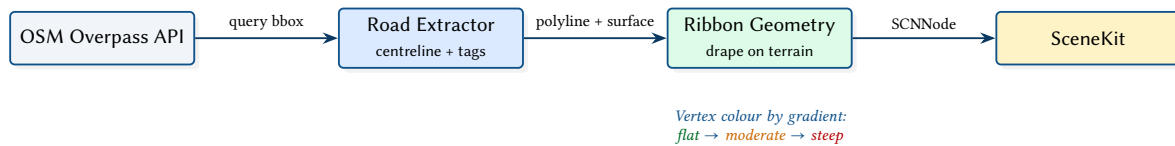


Figure 46. OSM road rendering pipeline. Road centrelines are extracted via the Overpass API, converted to ribbon geometry, and draped over the SRTM terrain mesh. Vertex colour encodes gradient steepness.

23.3 MapKit and Apple Maps Overlays

On iOS and tvOS, MapKit provides satellite imagery tiles, 3D terrain (via MKMapView’s globe mode), and point-of-interest data. A split-screen view could show the SceneKit terrain on one half and a MapKit satellite view on the other, with the rider’s position synchronised on both. This gives geographic context (“I am climbing the Col du Galibier”) without requiring the SceneKit renderer to reproduce photorealistic terrain.

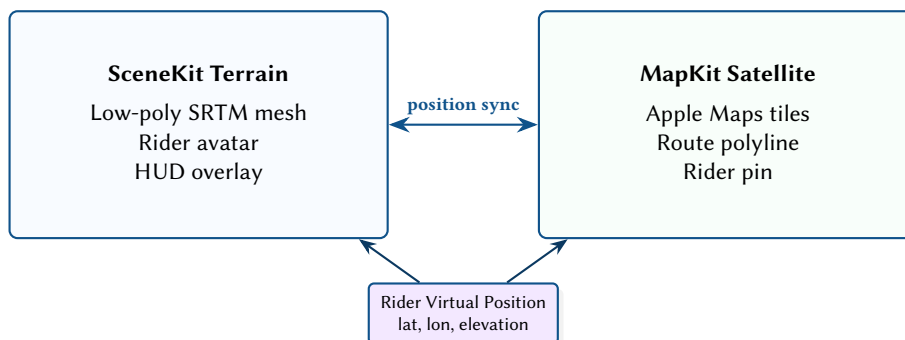


Figure 47. Split-screen geographic context. The left panel renders the SRTM-derived SceneKit terrain; the right panel shows Apple Maps satellite imagery. Both views track the rider’s virtual position, giving geographic context without requiring photorealistic 3D rendering.

23.4 Famous Climb Database

A curated library of the world’s most famous cycling climbs—Alpe d’Huez, Mont Ventoux, Stelvio Pass, Passo dello Stelvio, Sa Calobra, Haleakalā—each with pre-downloaded SRTM tiles, OSM road data, and a gradient profile. The rider selects a climb from a catalogue; the system loads the geographic data, sets the agent to replay the gradient profile, and renders the real terrain. This is the indoor cycling equivalent of “ride the Tour de France stages.”

23.5 Offline Tile Preparation

All geographic data can be pre-downloaded and cached locally. A companion macOS tool (or a build-time script) would:

1. Accept a GPX route or a bounding box.
2. Download SRTM tiles, OSM road data, and optionally satellite imagery tiles for the area.
3. Package them into a compact binary format optimised for sequential chunk loading.
4. Store the package on-device (via iCloud sync or direct transfer).

At ride time, no network access is required. The terrain generator reads from the local cache exactly as it would from a procedural seed. This is critical for Apple TV, which may not have reliable internet access in a garage or basement gym.

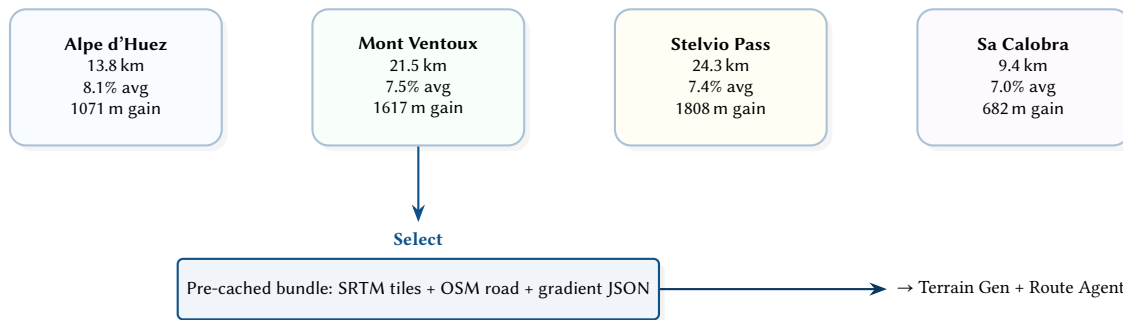


Figure 48. Famous climb catalogue. Each entry is a pre-cached bundle of SRTM elevation data, OSM road geometry, and a gradient profile. The rider selects a climb and the system loads the real-world terrain and gradient sequence.

: Offline-First Geographic Data

All real-world terrain data must be pre-cached on-device before the ride starts. The render loop must never block on a network request. Missing tiles are filled with procedural noise rather than showing blank terrain.

23.6 Terrain Fidelity Levels

Different data sources offer different resolutions. The system should support a fidelity ladder:

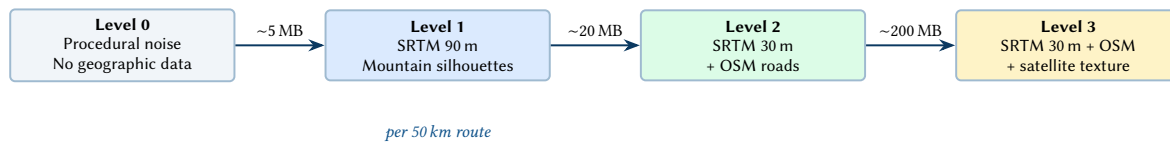


Figure 49. Terrain fidelity ladder. Each level adds more geographic detail at the cost of storage. Level 0 requires no download; Level 3 provides photorealistic terrain with satellite textures. The rider chooses their fidelity based on available storage and connectivity.

The key insight is that Level 0 (procedural) is always available as the fallback. If a tile is missing from the cache, the generator substitutes noise-generated terrain seamlessly. The rider never sees a loading screen or a blank chunk.

23.7 Street View Panoramas and Video Routes

Beyond 3D terrain, the system could display geo-tagged video footage or Street View-style panoramas synchronised to the rider's position. Services like CyclingCols and BikeMap provide first-person cycling video for famous routes. The video playback speed would be synchronised to the rider's virtual speed: pedal faster, the video advances faster.

This is a fundamentally different rendering mode—video playback rather than SceneKit—but the data flow is identical. The VisualSpeedMapper produces a position; that position indexes into a geo-tagged video timeline instead of a terrain chunk. The HUD overlays identically on both rendering modes.

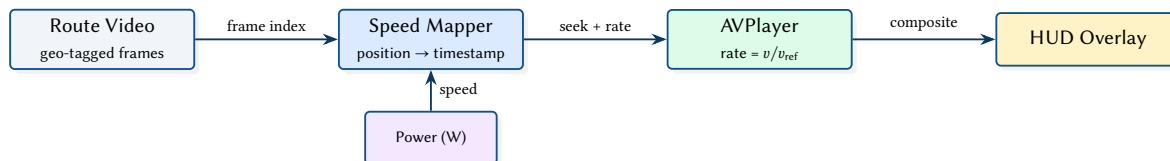


Figure 50. Video route playback. A geo-tagged cycling video replaces the SceneKit terrain. The speed mapper converts power to a playback rate: higher power advances the video faster. The HUD overlays identically on both SceneKit and video rendering modes.

24 Performance Considerations

The target is 60 fps on Apple TV, which gives a 16.6 ms frame budget. The current render loop performs three operations per frame:

1. **Speed update** (<0.1 ms): Integrate power into position.
2. **Chunk management** (~0.5 ms amortised): Check whether new chunks need generation. Chunk generation itself (simplex noise over a vertex grid) costs ~5 ms but occurs only when crossing a chunk boundary (every ~100 m of virtual travel).
3. **Camera interpolation** (<0.1 ms): Update harness position.

SceneKit handles the actual GPU rendering. The risk is that procedural geometry generation on chunk boundaries causes a frame spike. Mitigations include: generating the next chunk one frame early (predictive generation), moving generation to a background thread (the geometry is constructed on a worker and attached to the scene on the main thread), or reducing vertex density on Apple TV (where the viewing distance makes fine detail invisible).

24.1 NATS Message Throughput

At steady state, the system processes approximately:

- 4 BLE notifications/second (Indoor Bike Data at 250 ms intervals).
- 1 heart rate sample/second (Watch optical sensor cadence).
- 1–2 agent telemetry messages/second (agent-defined cadence).
- 0–2 agent commands/second (only during grade changes).

Total: fewer than 10 messages/second. NATS is designed for millions of messages per second. The system is approximately five orders of magnitude below the broker’s capacity. Message throughput will not be a bottleneck at any realistic scale.

24.2 Memory Pressure from Terrain Chunks

Each terrain chunk contains a vertex buffer (~32 KB for a 64×64 grid at 8 bytes per vertex position) plus normals and indices (~64 KB total). With 5 active chunks, the terrain consumes ~320 KB of GPU memory. This is negligible on iPad or Apple TV. However, if chunk density or resolution increases (for close-up detail), memory pressure should be monitored.

24.3 Apple Watch Battery Impact

Continuous heart rate sampling at 1 Hz via `HKAnchoredObjectQuery` during an active `HKWorkoutSession` is the standard HealthKit pattern and is optimised by watchOS. Apple Watch typically sustains 4–5 hours of continuous workout recording. The WatchConnectivity mirror transfer adds negligible overhead (one small dictionary per second). The Watch app should remain within normal workout battery expectations.

24.4 CADisplayLink vs. Timer Render Loop

On iOS and tvOS, the render loop uses `CADisplayLink`, which fires at the display’s refresh rate and provides a hardware-synchronised timestamp. On macOS (used during development), `CADisplayLink` is unavailable; a `Timer` at 1/60 s is used instead. The `Timer` path has higher jitter (± 2 ms) but is acceptable for development. The production path (iOS/tvOS) should always use `CADisplayLink`. ProMotion displays (120 Hz on iPad Pro) will automatically benefit, as `CADisplayLink` adapts to the display’s native refresh rate.

25 Summary

Cycle Agent decomposes the indoor cycling training problem into four independent roles—Display, Bridge, Sensor, and Agent—connected by a NATS broker. This decomposition allows each role to evolve independently, to be tested in isolation, and to run on the most appropriate hardware. The implementation is approximately 70% complete, with core connectivity, agent protocol, terrain rendering, and heart rate modelling operational. The primary remaining work is the Watch and Bridge integration (Phase 4) and the tvOS polish pass (Phase 5).

The analysis sections of this report reveal two categories of gap. **Robustness gaps:** the system lacks staleness detection, failure recovery, telemetry persistence, and crash resilience across all four links. The failure mode table (table 1) and the proposed Staleness Visibility design rule provide a framework for addressing these systematically. **Intelligence gaps:** the protocol infrastructure exists but no real training agent does. The agent decision engine, workout schema, and training plan progression represent the largest body of work remaining, and all of it can be developed offline.

The product and experiential ideas—workout builder, live dashboards, social riding, GPX route replay, haptic zone transitions, Metal shader effects, SharePlay group rides, cardiac drift detection, digital twin simulation, and a third-party agent marketplace—are enabled by the NATS subject namespace and the versioned envelope protocol. The majority require no changes to the existing Host, Bridge, or Watch applications. This is the architectural payoff of the role–device separation: new capabilities are new subscribers, not new features embedded in existing code.

Multiple components—workout schemas, agent decision logic, terrain themes, ride recording formats, protocol conformance tests, and onboarding UI prototypes—can be developed entirely offline, without hardware, to prepare for the final integration phases.

Revision History

Version	Date	Changes
0.1.0	March 2026	Initial architecture report